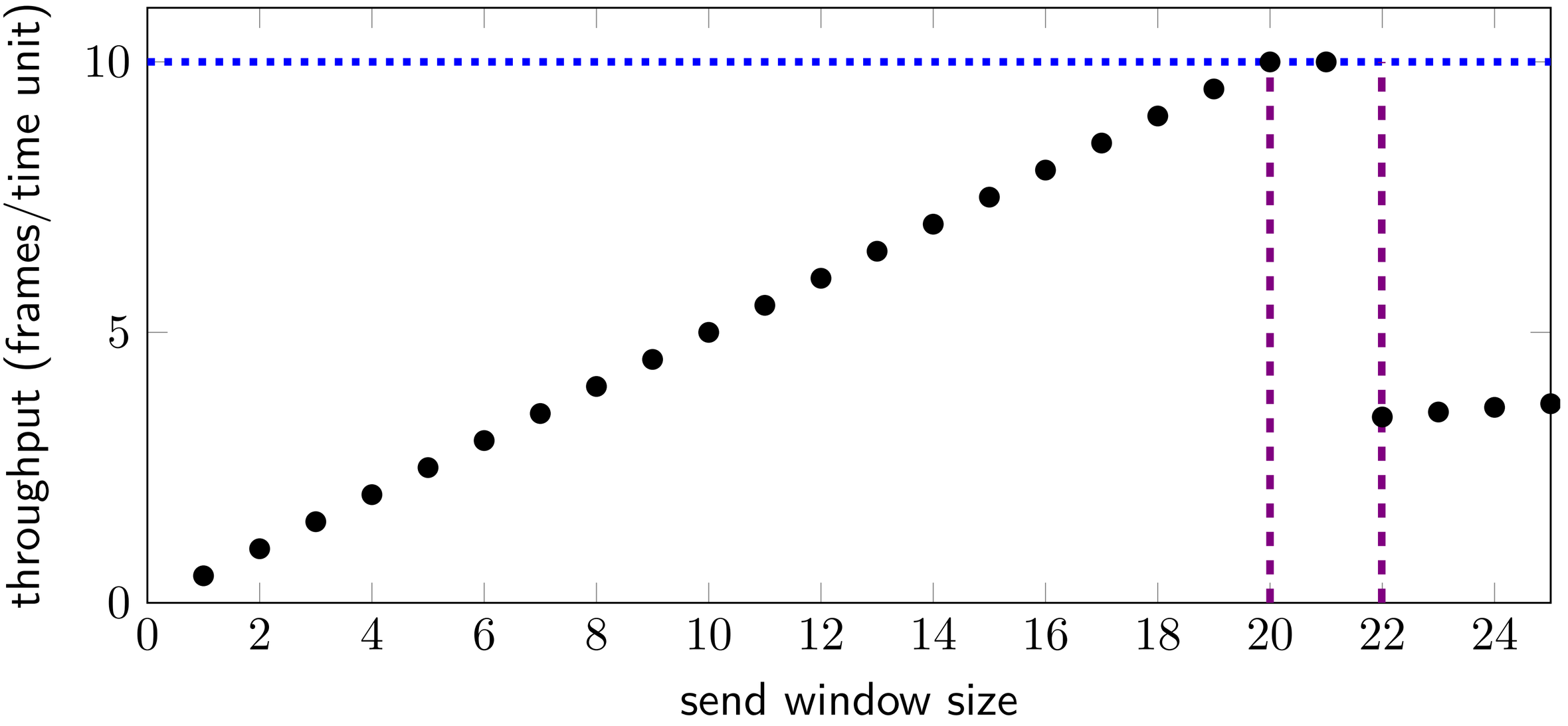
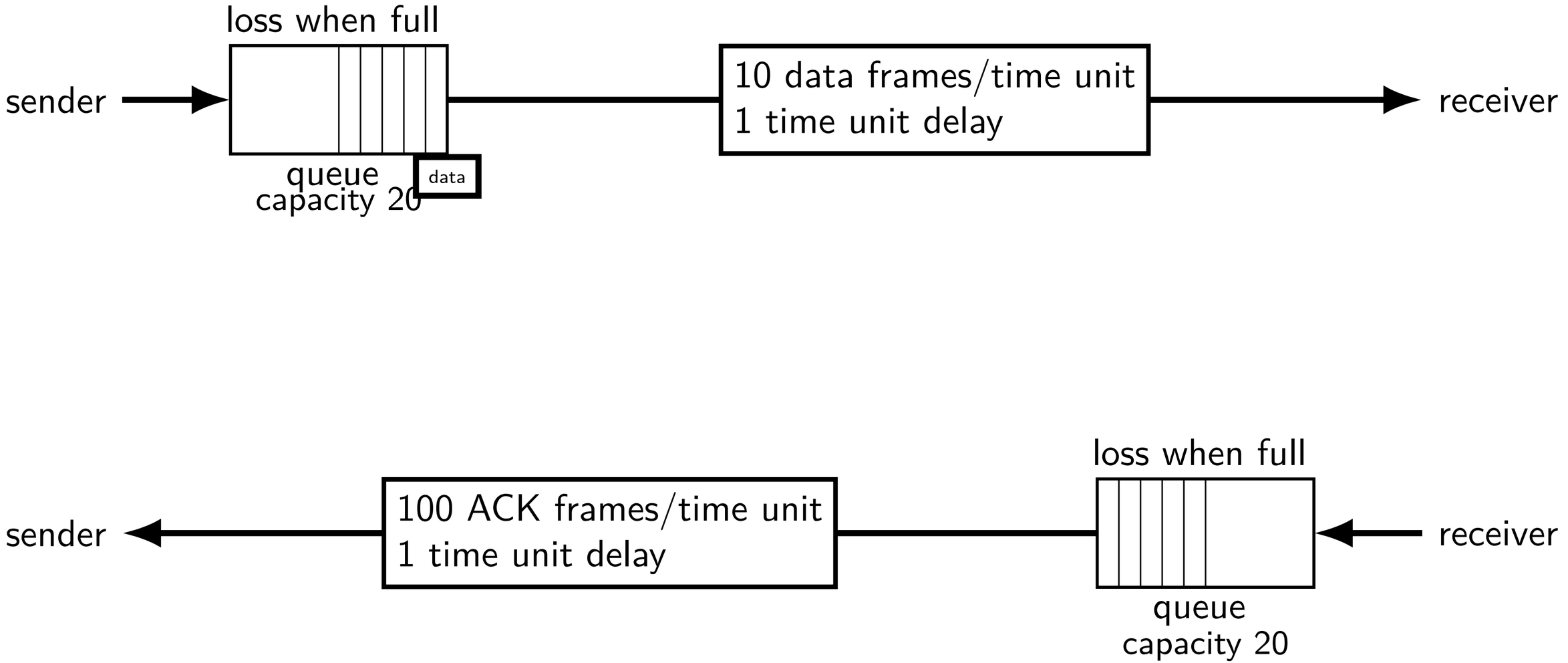
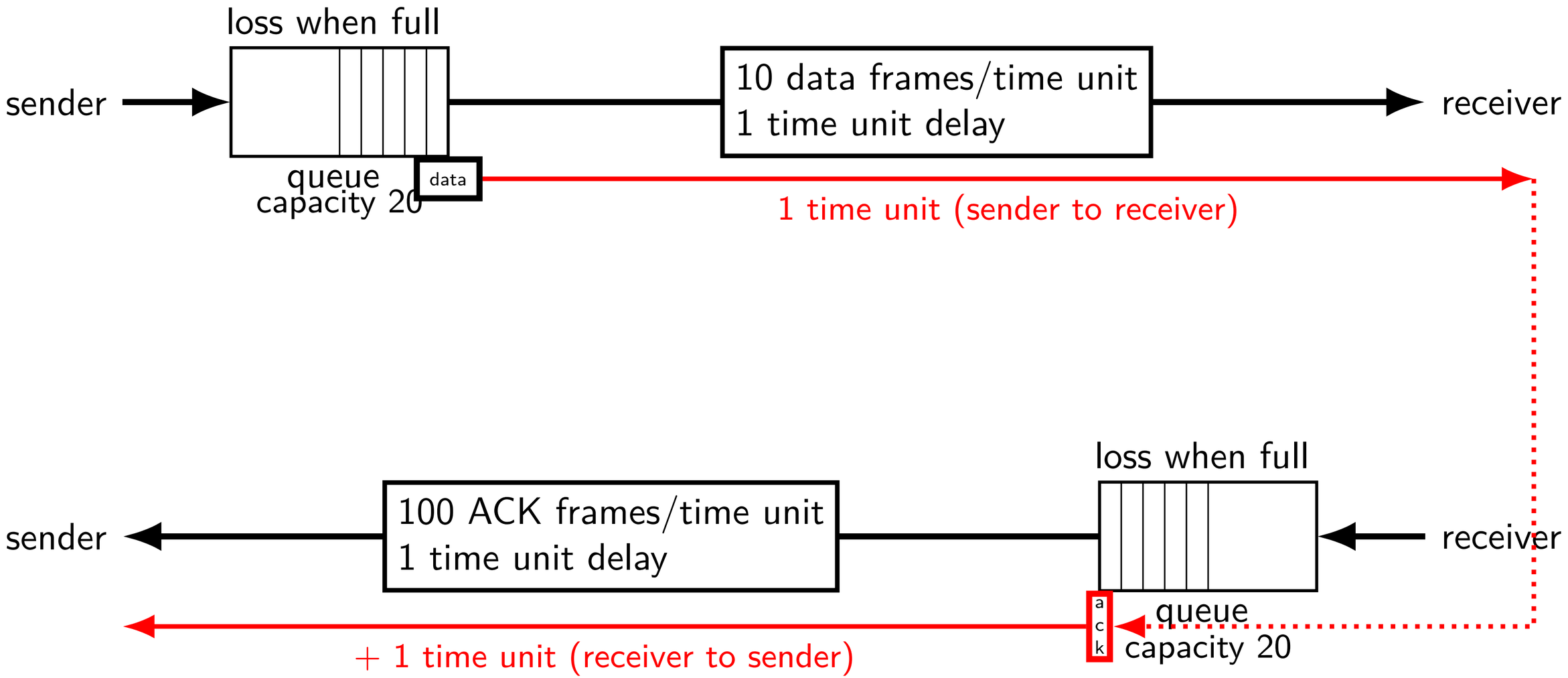
throughput and window size



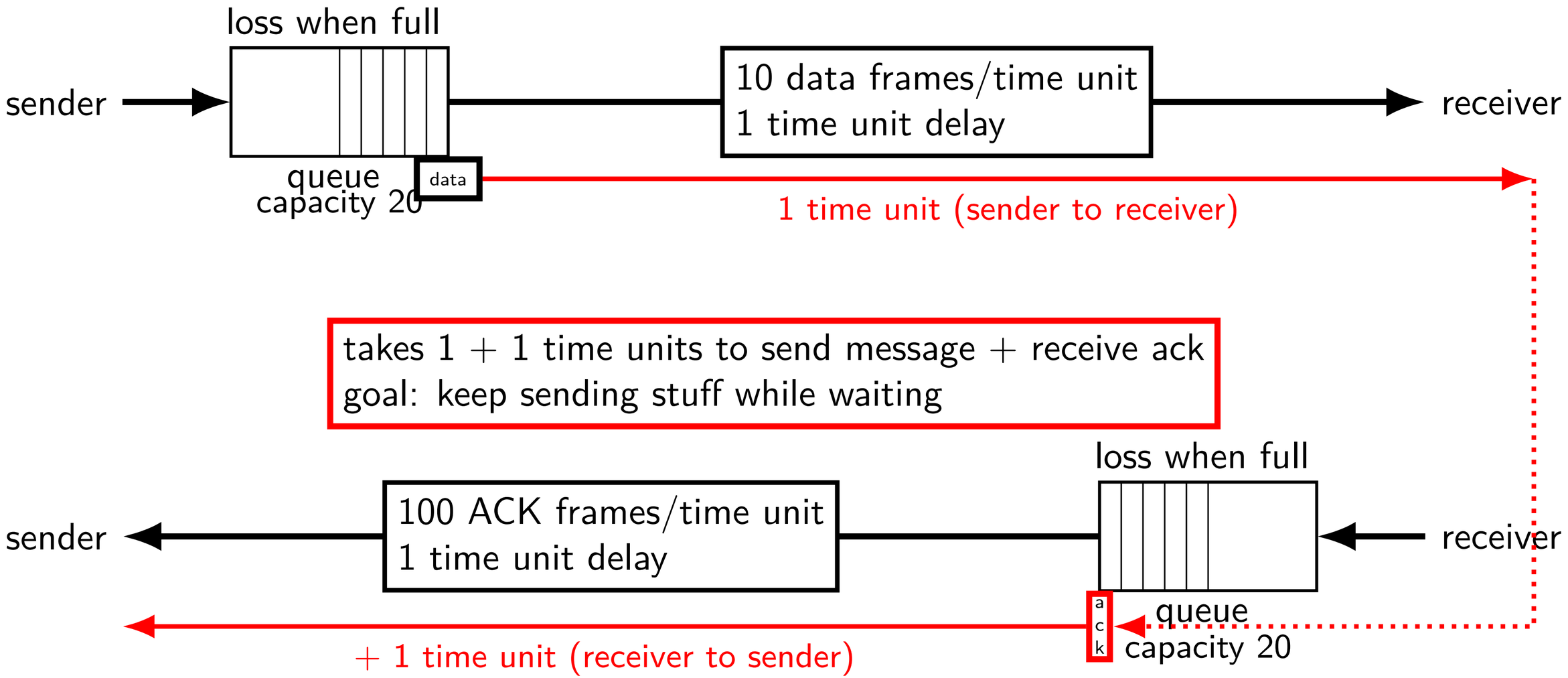
packet transit time



packet transit time



packet transit time



filling the pipe

round-trip time of 2 time units

from send data to receive ACK (assuming no queuing delay)

can send 10 data frames per time unit

= can send 20 data frames while waiting for ACK

“bandwidth-delay product”

10/time unit (banwidth) times 2 time unit (RTT = delay)

why optimal

window size $\sim$ number of packets in flight

in normal operation with window size W

- receive ACK for x (now $W - 1$ in flight)
- send packet $x + W$
- receive ACK for $x + 1$
- send packet $x + W + 1$
- ...

window size keeps W packets in flight

- increase window = more packets in flight
- decrease window = reduce packets in flight

ideal choice $\sim$ amount network can handle

number in flight on losses

window size W

let's say we lose packet x [only], sender might

- receive ACK for $x - 1$
- send packet $x + W - 1$
- receive ACK for $x, x, x, \ldots$
- resend packet x (guess it is lost)
- receive ACK for $x, x, x, \ldots$
- receive ACK for packet $x + W - 1$
- send packets $x + W$ through $x + W + W - 1$

number in flight on losses

window size W

let's say we lose packet x [only], sender might

- receive ACK for $x - 1$
- send packet $x + W - 1$
- receive ACK for $x, x, x, \ldots$
- resend packet x (guess it is lost)
- *receive ACK for $x, x, x, \ldots$*
- receive ACK for packet $x + W - 1$
- send packets $x + W$ through $x + W + W - 1$

number in flight on losses

window size W

let's say we lose packet x [only], sender might

receive ACK for $x - 1$

send packet $x + W - 1$

receive ACK for $x, x, x, ...$

resend packet x (guess it is lost)

receive ACK for $x, x, x, ...$

receive ACK for packet $x + W - 1$

send packets $x +$

lots of time when we are not sending packets

means links are underutilized

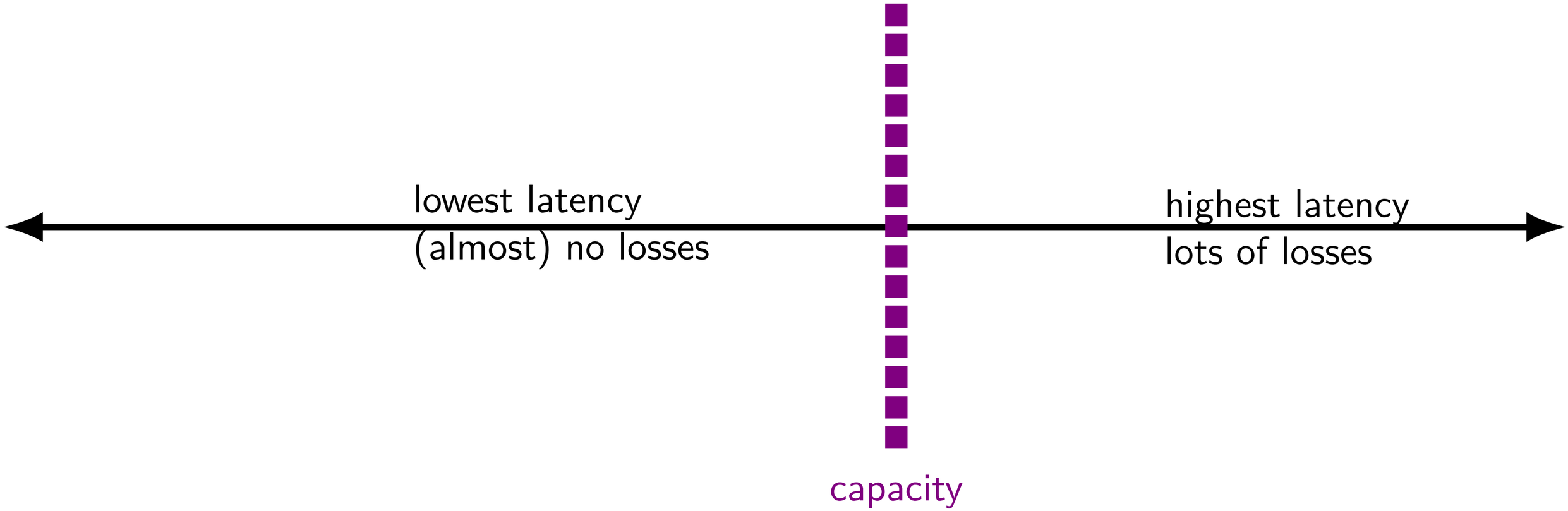
window size tweaking

window size imperfect proxy for # packets in flight

we'll ignore the difference for now

our goal for now: window size = number of packets to have in flight

finding window size empirically (1)



key insight

latency/loss rate increases when window size too big

latency/loss rate stable when window size not too big

for now, we'll focus on loss rate

- but you can do something similar with latency

try a bunch of things

try a bunch of things

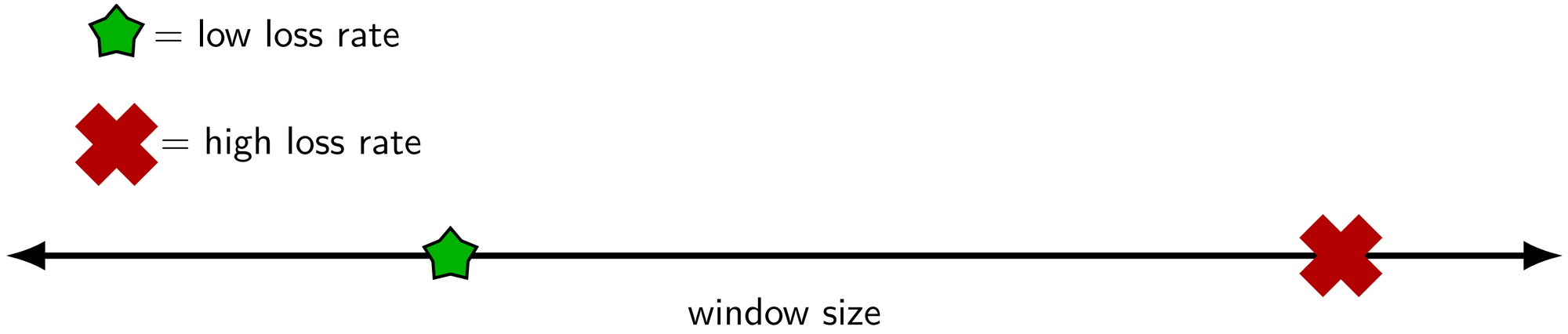

= low loss rate


= high loss rate

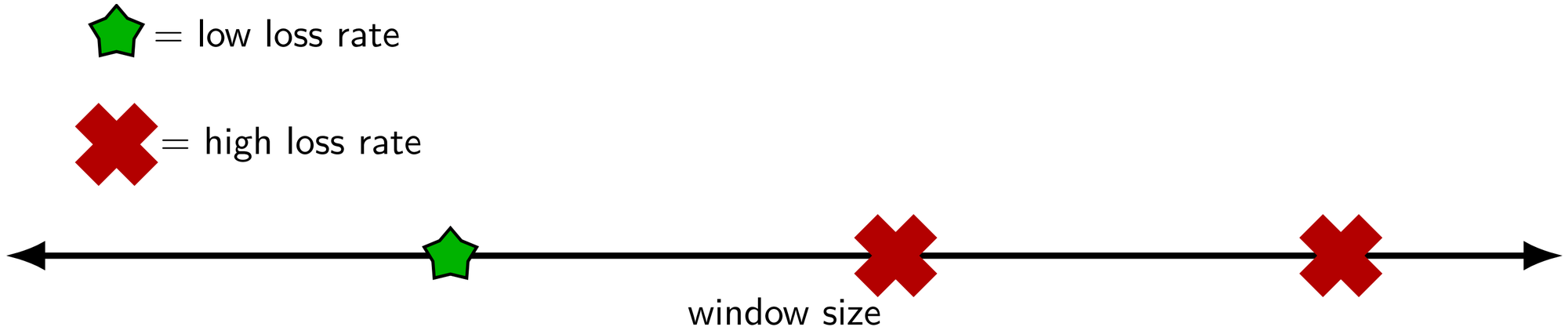


window size

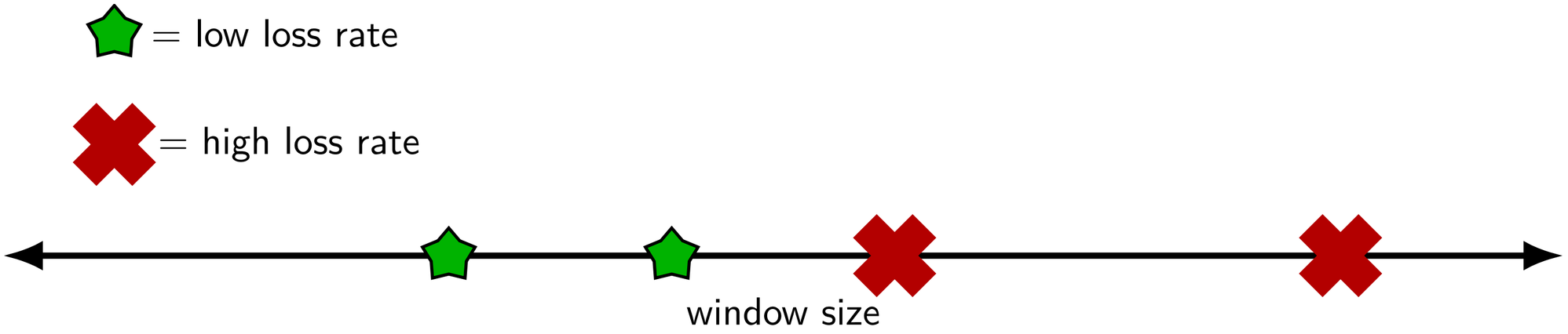
try a bunch of things



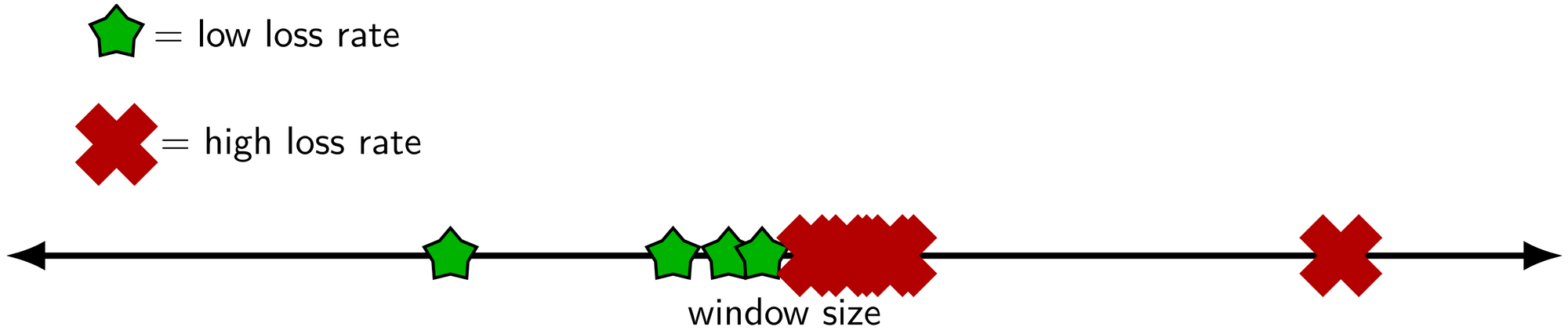
try a bunch of things



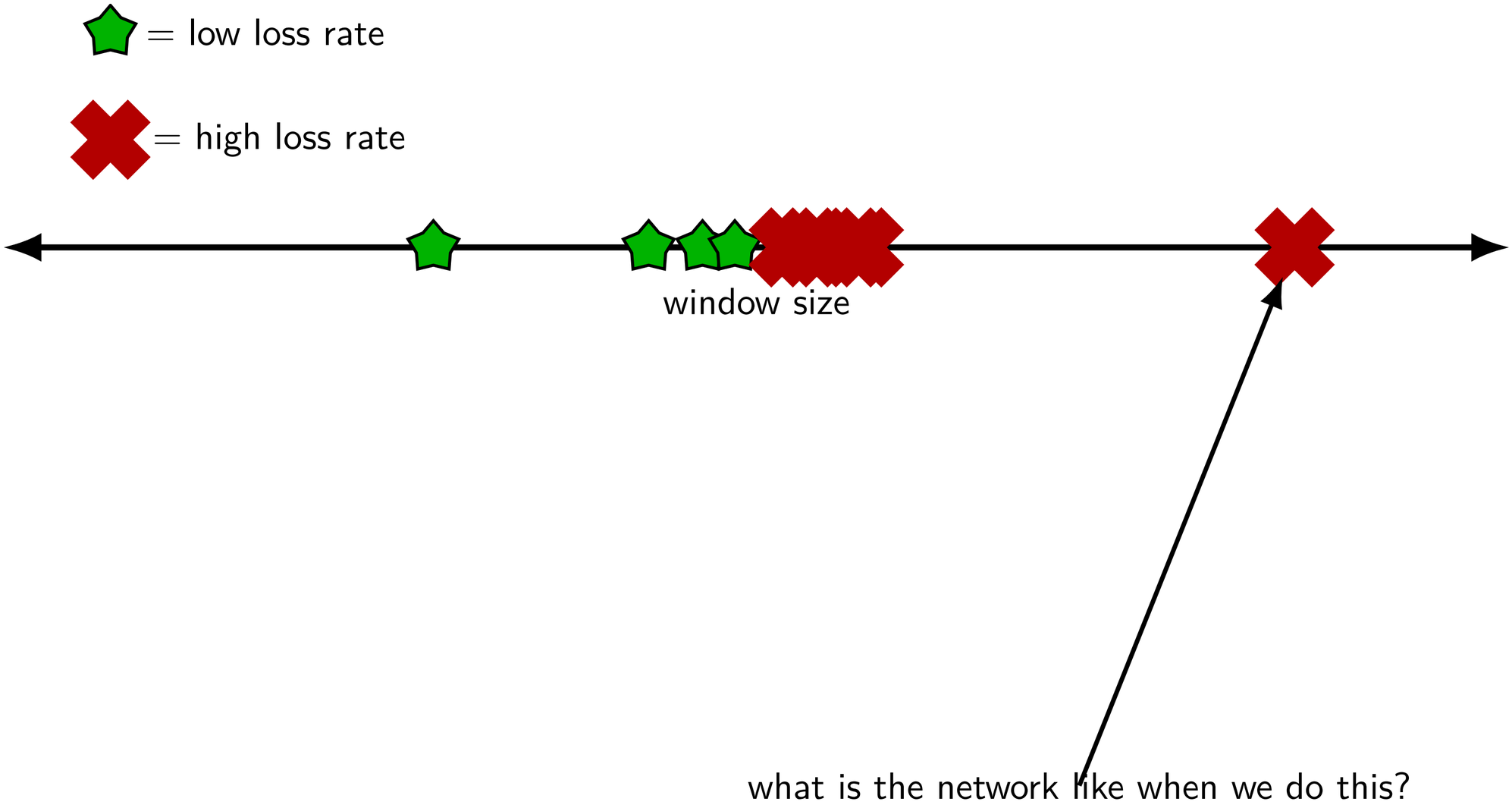
try a bunch of things



try a bunch of things



try a bunch of things



revisiting congestion collapse

Congestion Avoidance and Control

Van Jacobson*

University of California
Lawrence Berkeley Laboratory
Berkeley, CA 94720
van@helios.ee.lbl.gov

In October of '86, the Internet had the first of what became a series of 'congestion collapses'. During this period, the data throughput from LBL to UC Berkeley (sites separated by 400 yards and three IMP hops) dropped from 32 Kbps to 40 bps. Mike Karels[1] and I were fascinated by this sudden factor-of-thousand drop in bandwidth and embarked on an investigation of why things had gotten so bad. We wondered, in particular,

fixes from Jacobson's 1987 paper

1. round-trip-time variance estimation
2. exponential retransmit timer backoff
3. slow-start
4. more aggressive receiver ack policy
5. dynamic window sizing on congestion
6. Karn's clamped retransmit backoff
7. fast retransmit

the overloaded switch

let's say switch can handle 50 packets/second

but has:

- 100 packets/second from test flow sending as fast as it can
- 10 packets/second from other session

expected *loss rate* (% packets lost)?

expected % test flow packets lost?

expected other session packets lost?

modeling who gets dropped

it kinda does matter…

sending in big bursts or spread out (“pacing”)?

- bursts can overload queues even though average rate is low

how switch’s queue works?

- queue size (handling bursts), way to choose what to drop

random or fixed intervals between sending?

but we’ll simplify, assuming—

- a flow’s arrivals are randomly spaced
- drops hit packets at random
- queue is “pretty big”

modeling who gets dropped

it kinda does matter…

sending in big bursts or spread out (“pacing”)?

- bursts can overload queues even though average rate is low

how switch’s queue works?

- queue size (handling bursts), way to choose what to drop

random or fixed intervals between sending?

but we’ll *simplify*, assuming—

- a flow’s arrivals are randomly spaced
- drops hit packets at random
- queue is “pretty big”

the overloaded switch

let's say switch can handle 50 packets/second

but has:

- 100 packets/second from test flow (checking window size)
- 10 packets/second from other session

expected *loss rate* (% packets lost)? $\frac{100+10-50}{100+10} = 54\%$

expected % test flow packets lost? 54%

expected % other session packets lost? 54%

… but I missed something

a virtuous cycle

what is other session going to when 54% of its packets are lost?

probably resend them

what about when resent packets are lost?

probably resent again

if other session doesn't slow down, then…

10 pkt/s $\rightarrow 10 + 54\% \cdot 10 + 54\%^2 \cdot 10 \ldots \approx 22$ pkt/s

the overloaded switch (revised)

let's say switch can handle 50 packets/second

but has:

- 100 packets/second from test flow (checking window size)
- 10 packets/second from other session $\rightarrow$ 22 with resends

expected *loss rate* (% packets lost)? $\frac{100+22-50}{100+22} = 59\%$

expected % test flow packets lost? 59%

expected % other session packets lost? 59%

- means that 22 pkt/sec is slight underestimate
- though realistically other session should slow down

aside: latency (1)

need one round-trip time (RTT) to detect loss

- probably from duplicate ACK
- if detecting via timeout, probably longer

so need 1.4 RTTs (detecting loss 1.4 times) extra time

mean latency $= \frac{1.4\text{RTTs}}{0.5\text{RTTs}}$ times normal $= 2.8$ times normal

aside: high-percentile latency

59% packet loss

about 10% of time need more than 4 retransmissions

about 5% of the time need more than 5 retransmissions

about 1% of the time need more than 8 retransmissions

sliding windows and retransmissions

we were assuming that other session doesn’t slow down

sliding window approach slows down on losses given same window size

sliding window throughput collapse

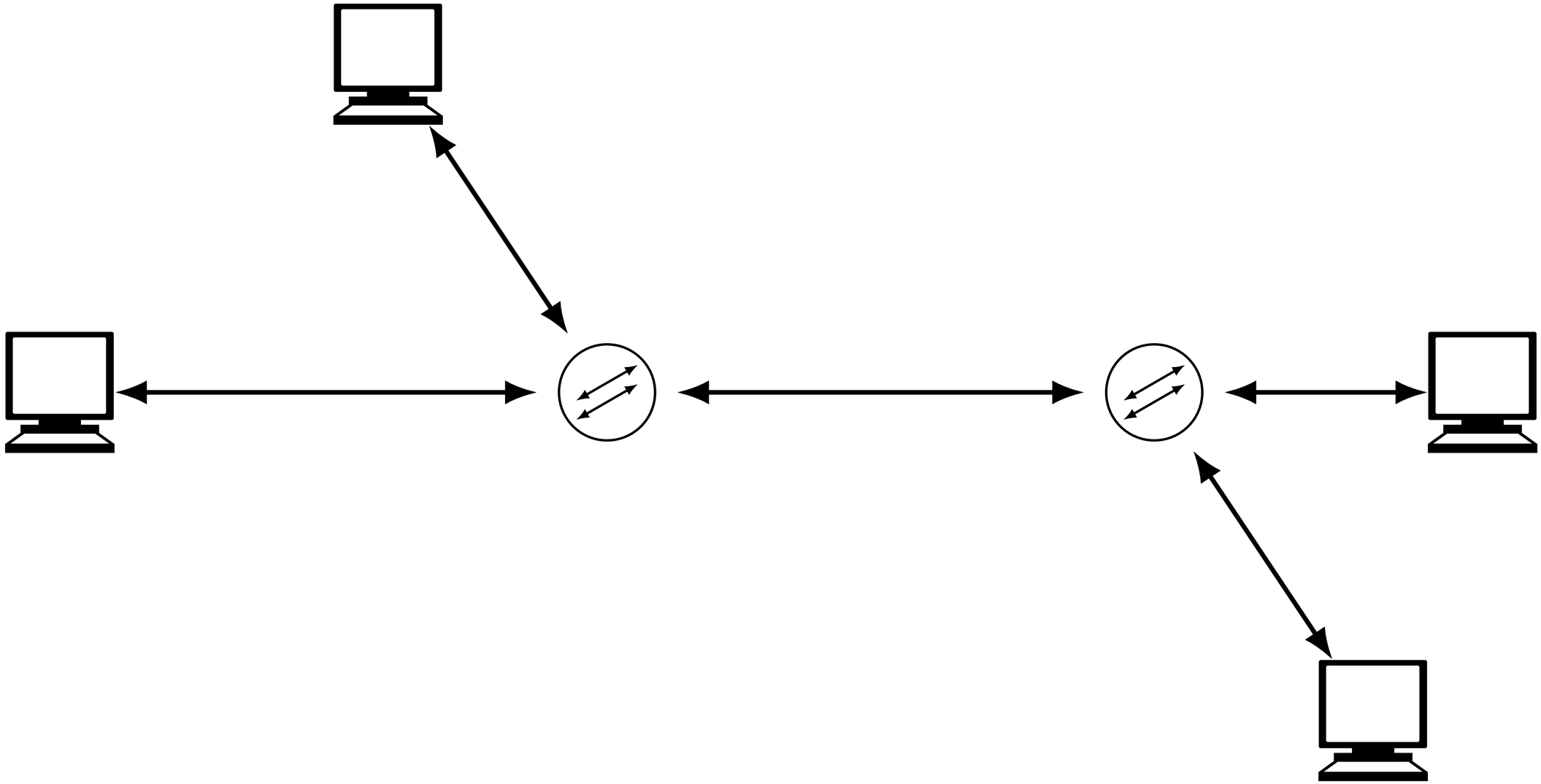
let's say doing sliding window with 100 packet window

if 1% of the time, we need to resend a packet 8 times, then

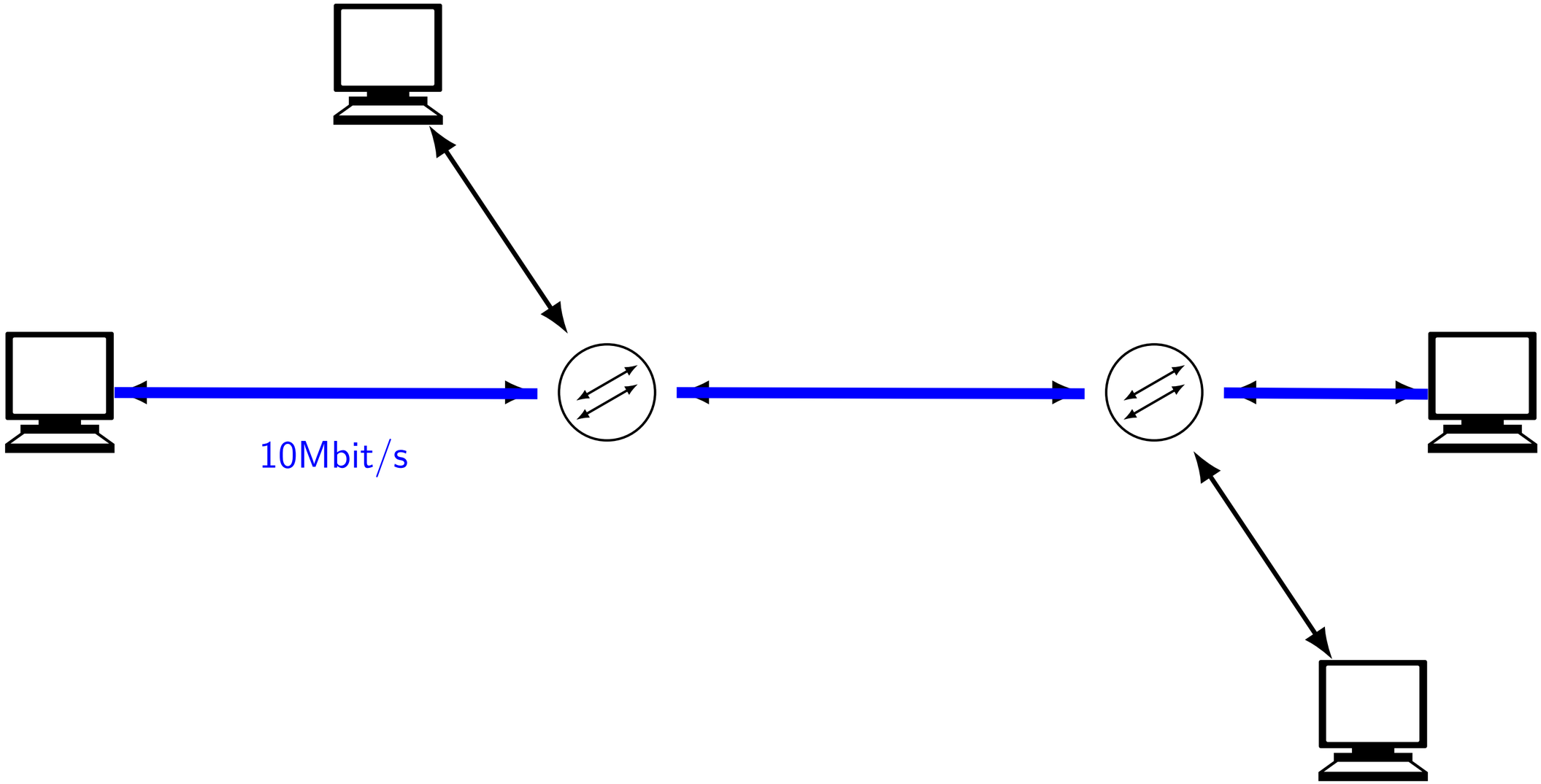
probably need around 8 RTTs to send all 100 packets in window

$\approx$ 8 times slower with same window size

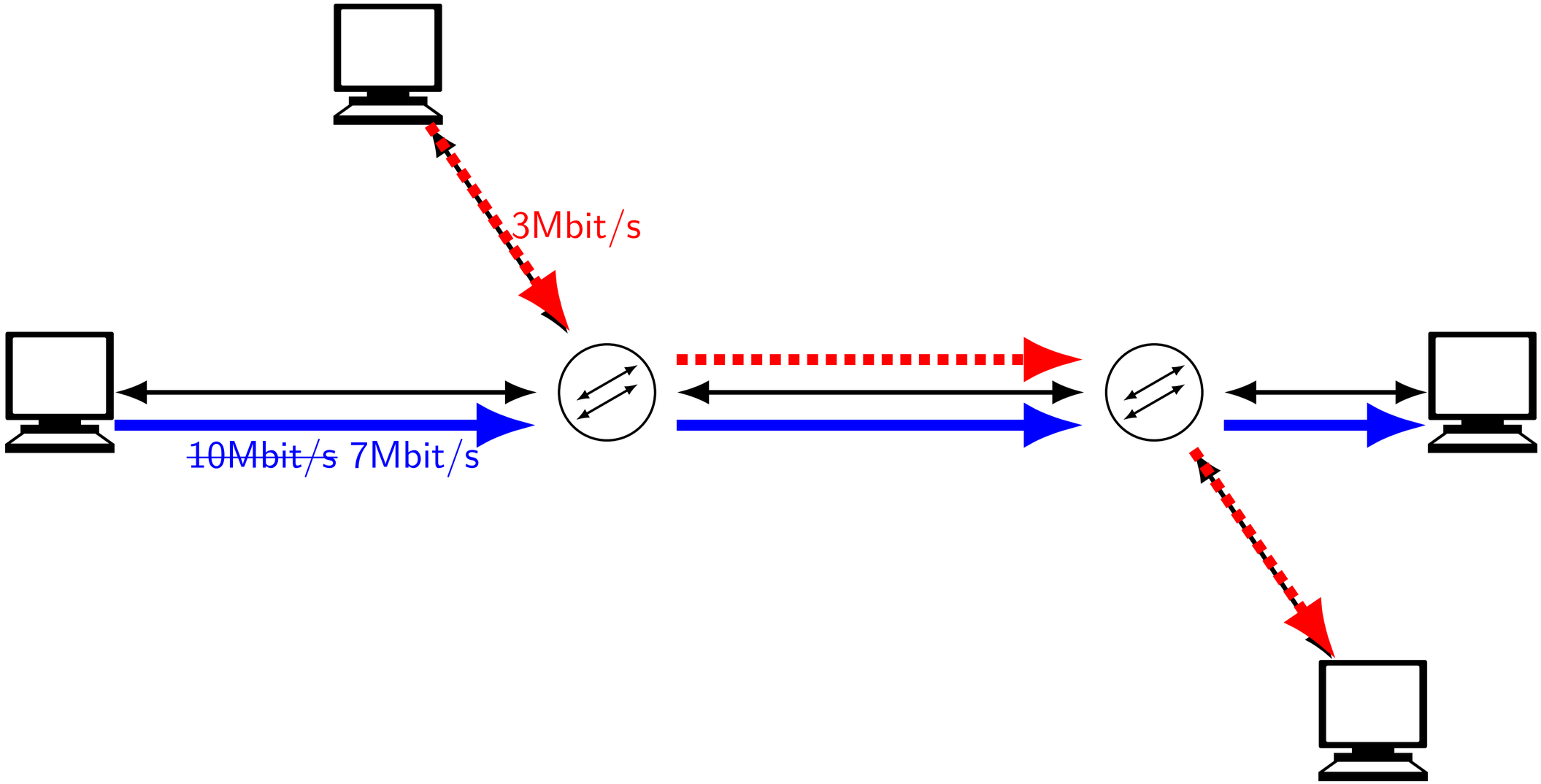
changing cross-traffic



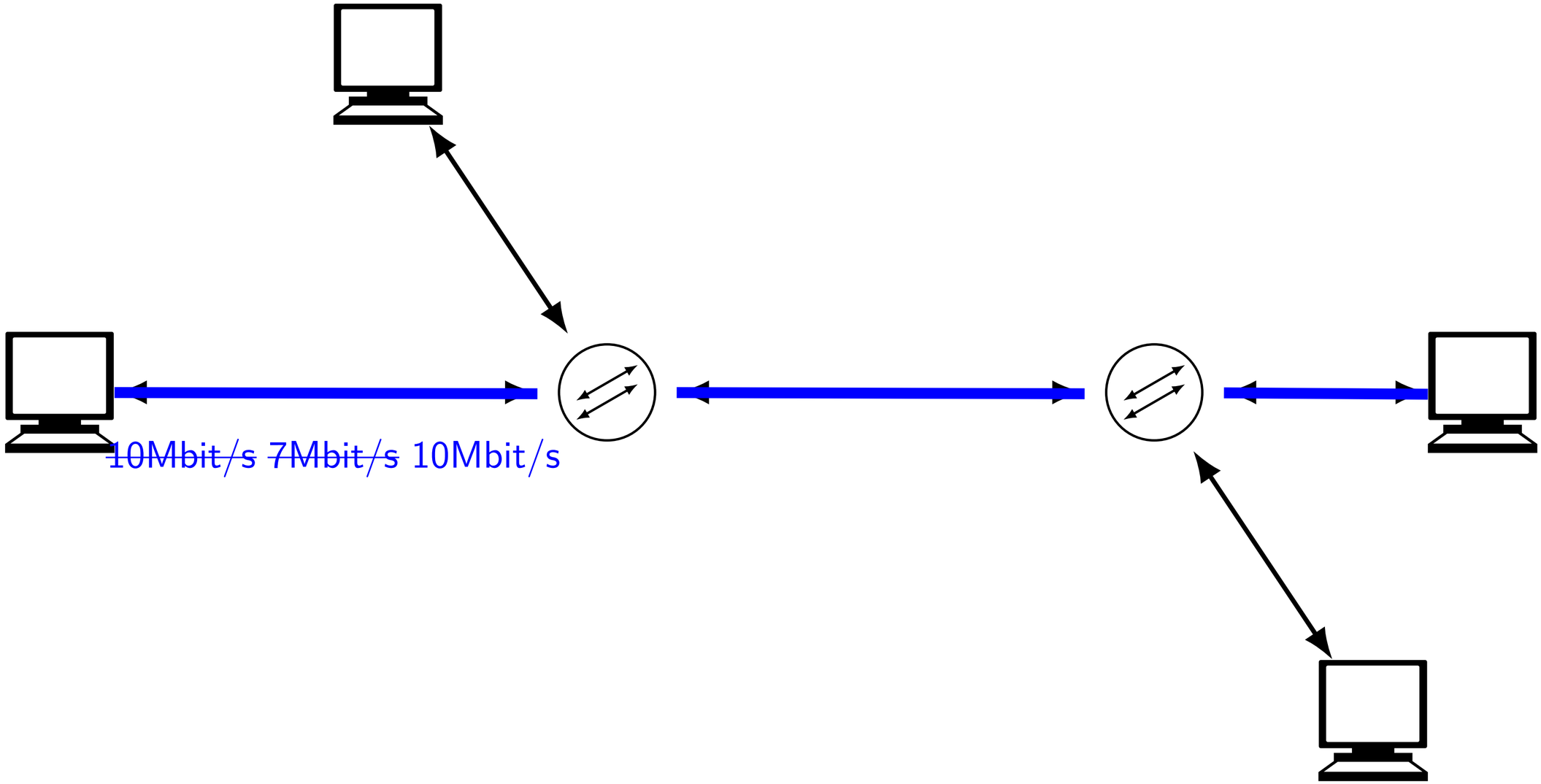
changing cross-traffic



changing cross-traffic



changing cross-traffic



adapting to cross-traffic

available bandwidth will change

previous example: 3Mbit lost/added from other flow

need to adapt to lost bandwidth

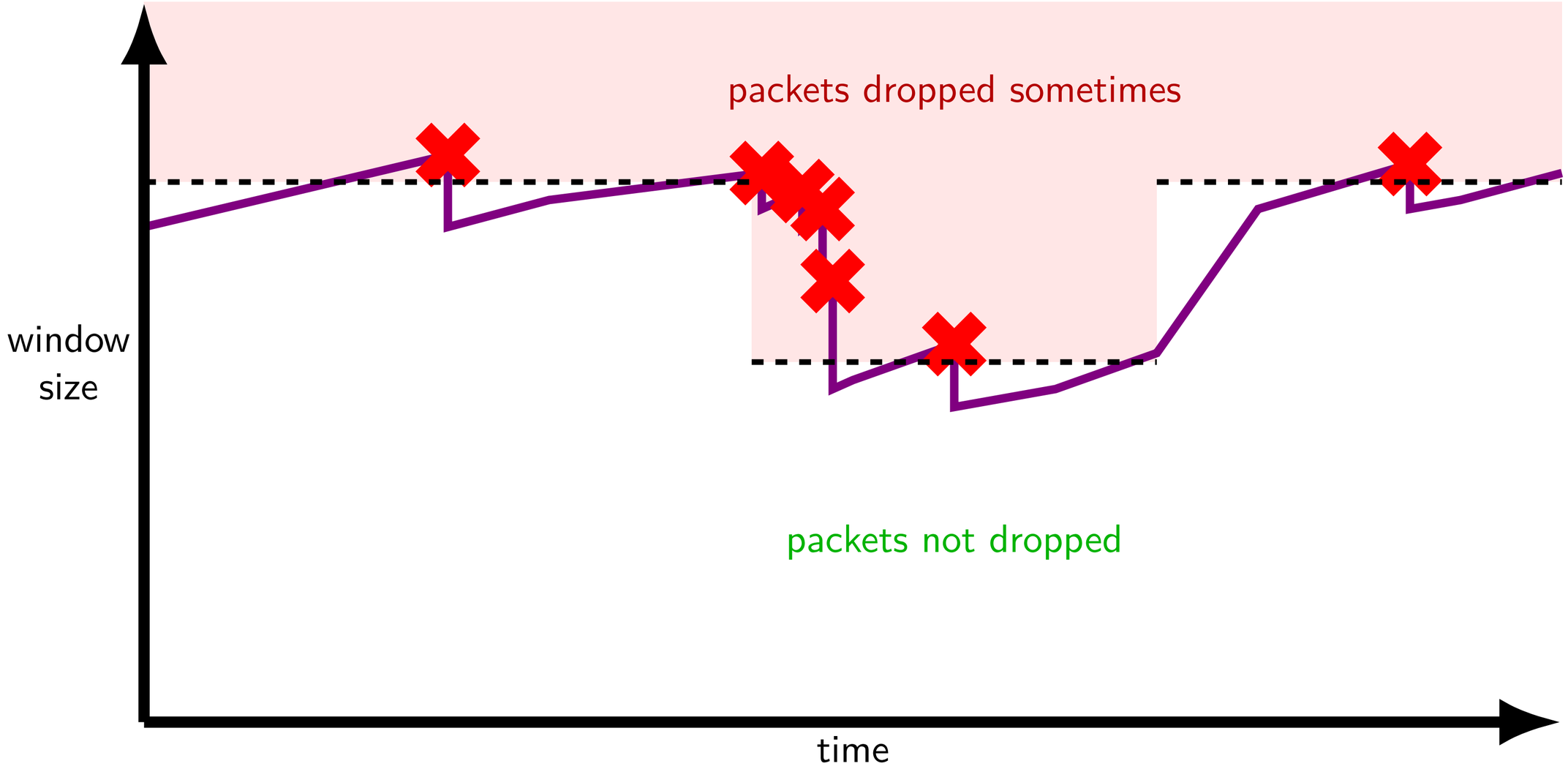
need to detect new available bandwidth

other flow's bandwidth?

for now, we'll pretend other flows don't react to us

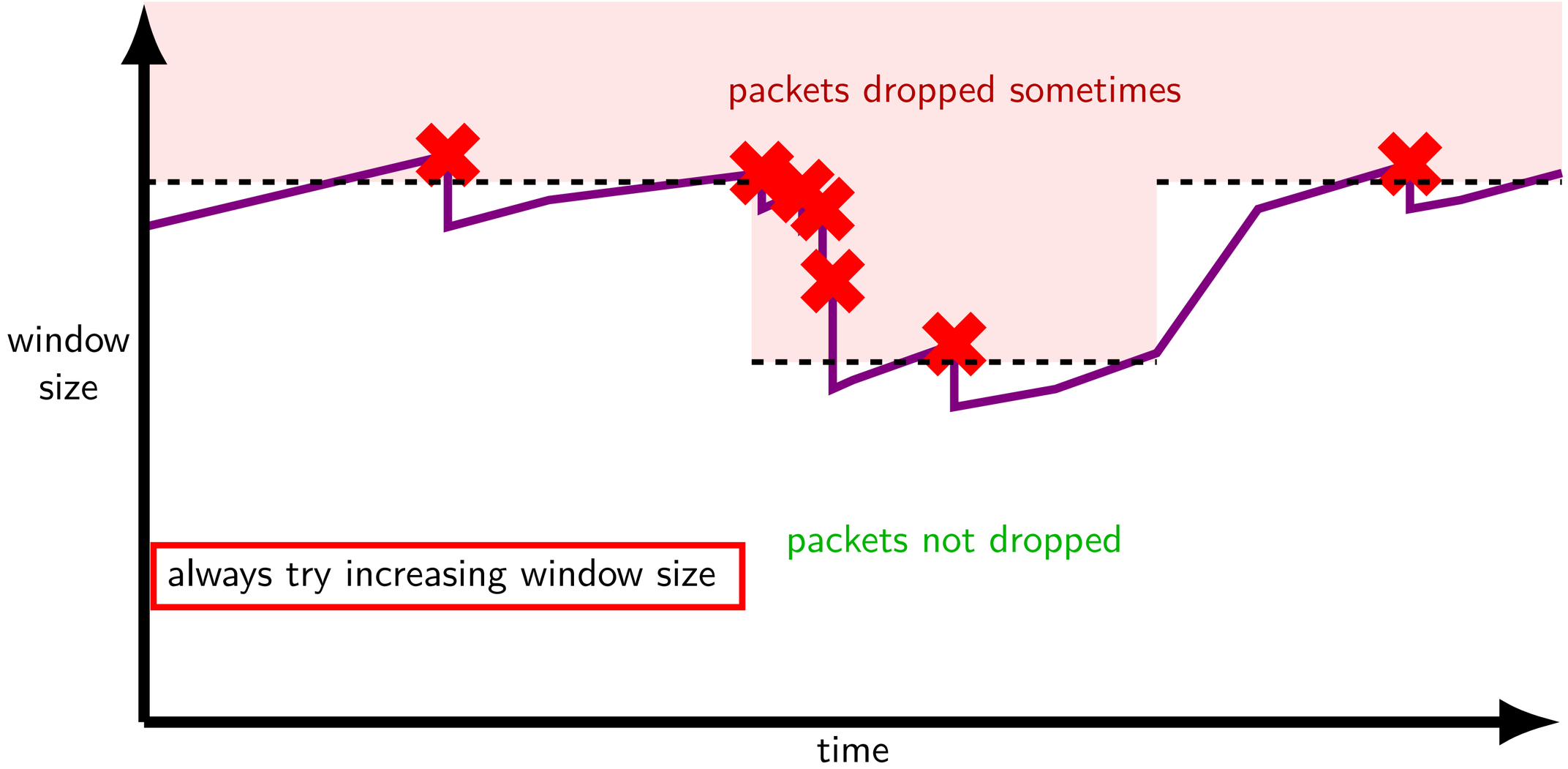
later topic: what happens when both reacting?

window size experimenting

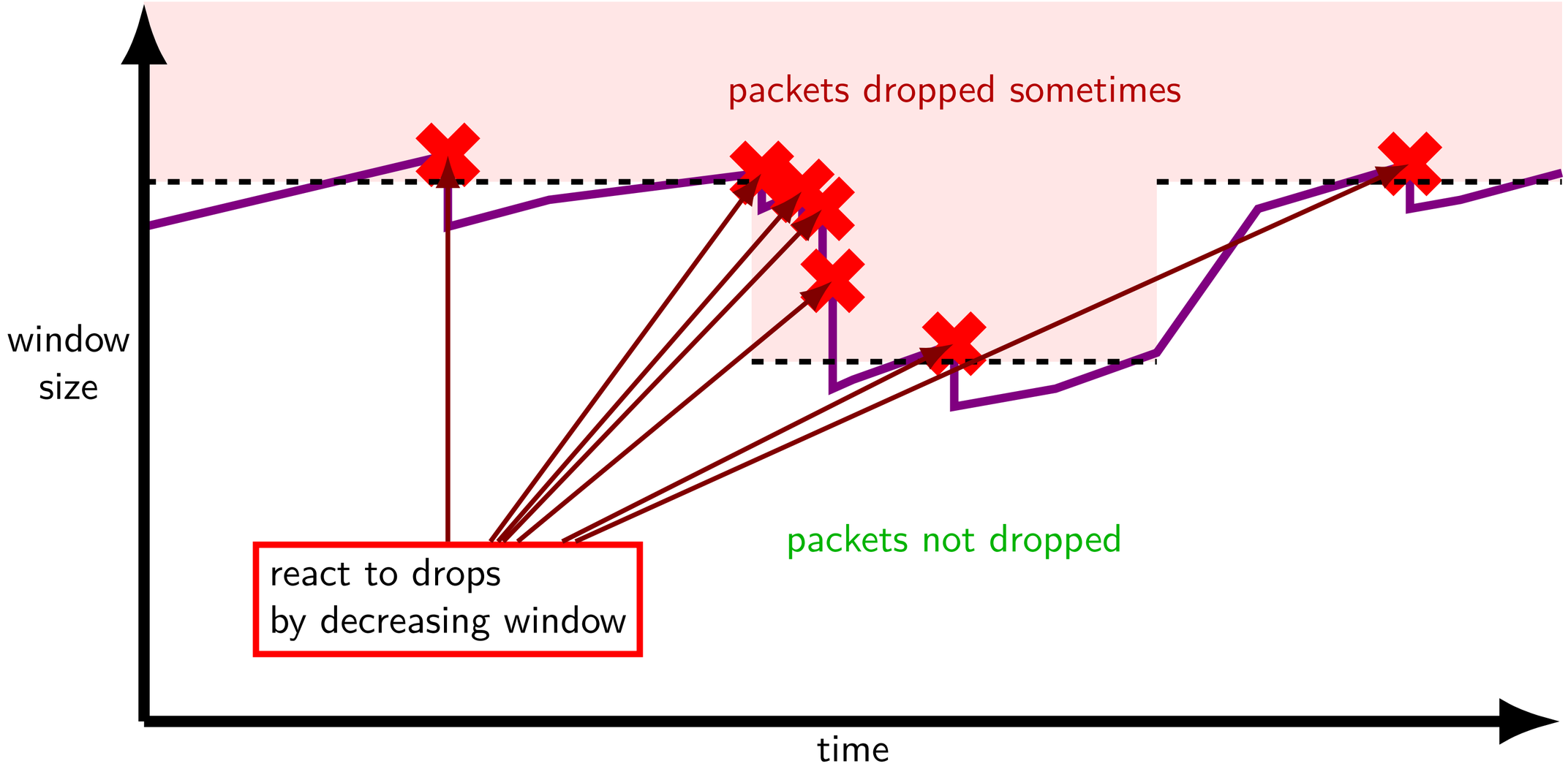


window size experimenting

window size experimenting



window size experimenting



increase/decrease strategy

default to increasing window size

react to packet drops by decreasing window size

- assumption: few “non-congestion” packet losses

big topic: how fast to do each?

questions to help decide that:

- what happens if we increase too fast? too slow?
- what happens if we decrease too fast? too slow?

increase/decrease strategy

default to increasing window size

react to packet drops by decreasing window size

assumption: few "non-congestion" packet losses

big topic: how fast to do each?

questions to help decide that:

what happens if we increase too fast? too slow?

what happens if we decrease too fast? too slow?

slow increase

want to increase *slowly* to avoid overload

original TCP: +1 packet/round trip time

+1 certainly not optimal choice, but okay heuristic

important theoretically: approx. *additive* increase

- helps ensure good behavior with multiple connections
- (we'll talk later about why)

exercise: convergence time (1)

suppose: 50 ms round trip time

initially sending at 600 packets/second

≈ 0.9Mbyte/sec with 1500 byte packets

optimal rate is 10000 packets/second

≈ 15Mbyte/sec with 1500 byte packets

‘standard’ TCP increase of 1 packet/RTT

how long to get there?

exercise: convergence time (1)

suppose: 50 ms round trip time

initially sending at 600 packets/second

≈ 0.9Mbyte/sec with 1500 byte packets

optimal rate is 10000 packets/second

≈ 15Mbyte/sec with 1500 byte packets

‘standard’ TCP increase of 1 packet/RTT

how long to get there?

current: 30 packets/RTT (= window size 30)

need to get to: 500 packets/RTT

will take $500 - 30 = 470$ round trips $\approx$ 23500 ms $\approx$ 24 s

fixing bad convergence time

TCP's additive increase is very slow for "high bandwidth-delay" networks

two things make this better:

not in additive increase mode at start of connection

- "slow start" we'll talk about later

more adaptive increase for modern TCP variants

- e.g. FAST TCP, CUBIC TCP, ...
- heuristics to increase faster when appropriate

heurstic: fast decrease (v1)

want to decrease quickly to get out of overload

original TCP heuristic: reset window to (minimum) 2 segments

TCP Tahoe (1)

first version of TCP congestion control

track two variables

- `cwnd` = congestion window
- `ssthresh` = 'slow start threshold'

three modes of operation:

congestion avoidance (cwnd $\geq$ ssthresh; no known loss)

recovery (known loss)

"slow start" (cwnd < ssthresh)

TCP Tahoe (2)

congestion avoidance (cwnd $\geq$ ssthresh; no known loss)

- represents "steady state" — our focus for now
- no loss: cwnd += 1 segment per round-trip-time
- on loss: cwnd = ~ 2 segments; ssthresh = cwnd/2

recovery (known loss)

- retransmit packet after third duplicate ACK of previous packet
- (or timeout)

"slow start" (cwnd < ssthresh)

- intuition: not in "steady state" yet
- want to get there quickly, but with overloading network
- we'll give details later

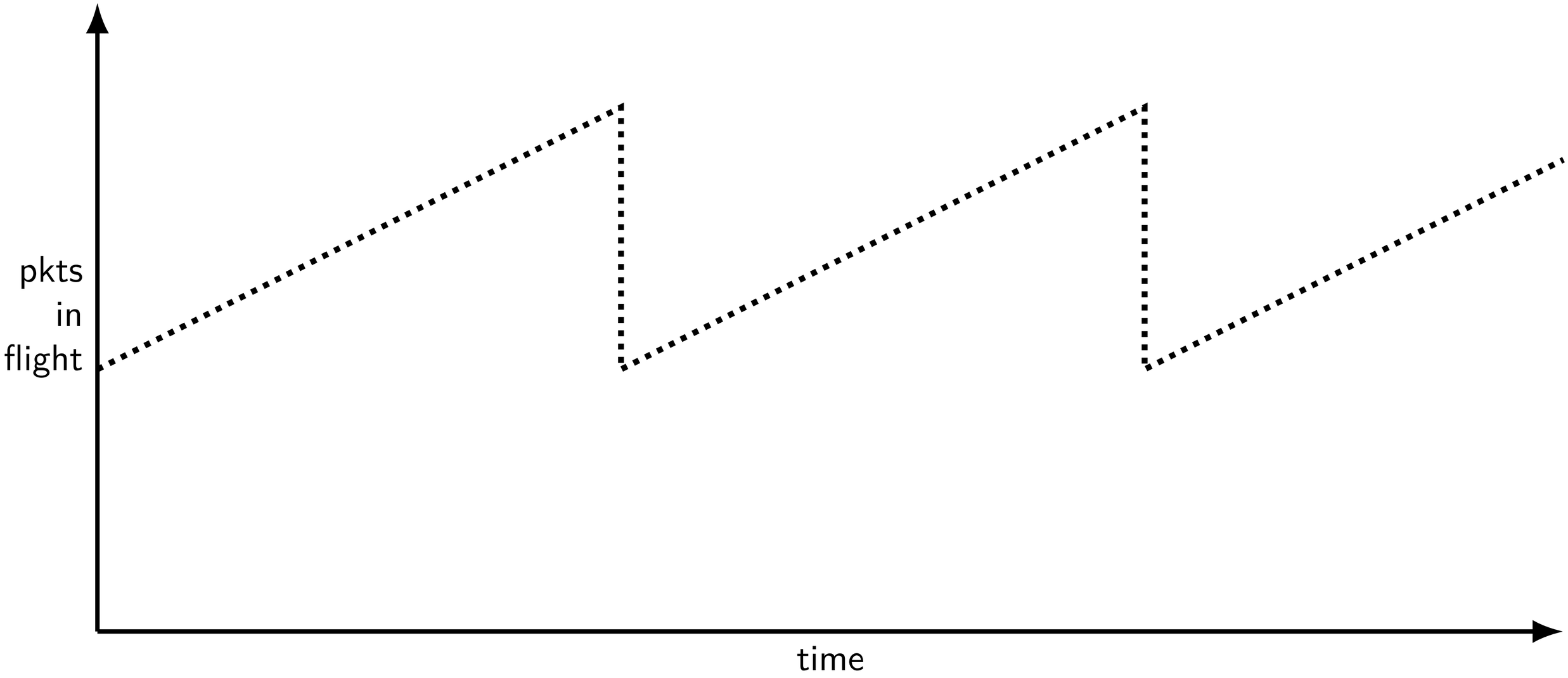
plan

really don’t want to reset to beginning on loss

- even if we have a way to get to good size quickly

would like to do less extreme decrease: cwnd = cwnd/2

sawtooth pattern



Linux's ss command

```
$ ss --info --numeric --tcp state established
...
Recv-Q  Send-Q   Local Address:Port          Peer Address:Port
0       2583232  128.143.71.27:5201          128.143.63.240:55926
cubic wscale:7,7 rto:204 rtt:1.891/0.125 ato:40 mss:1448 pmtu:1500
rcvmss:536 advmss:1448 cwnd:309 ssthresh:64 bytes_sent:152936312
bytes_acked:152698840 bytes_received:37 segs_out:105620 segs_in:7024
data_segs_out:105619 data_segs_in:1 send 1.89Gbps lastrcv:1312
pacing_rate 2.27Gbps delivery_rate 996Mbps delivered:105456
busy:1308ms unacked:164 rcv_space:14600 rcv_ssthresh:64076
notsent:2345760 minrtt:0.219
```

`cubic` = which congestion control algorithm

`cwnd` = 309 segments; `ssthresh` = 64 segments

cwnd = 309 segments $\times$ 1488B max segment size = 437 KiB

packet rate $\approx$ cwnd / rtt = 224 MiB/sec = 1879 Mbit/s

handling steady state

most of the time we should be at approx. correct window size

want to focus on how we react to changes

still going to use “experimentation” idea

fixes from Jacobson's 1987 paper

1. round-trip-time variance estimation
2. exponential retransmit timer backoff
3. slow-start
4. more aggressive receiver ack policy
5. *dynamic window sizing on congestion*
6. Karn's clamped retransmit backoff
7. *fast retransmit*

TCP (New)Reno

base for ‘modern’ TCP (changes from Tahoe highlighted)

congestion avoidance (cwnd $\geq$ ssthresh; no known loss)

- no loss: cwnd += 1 segment per round-trip-time
- *on loss (dup ACK): cwnd = cwnd/2; ssthresh = new cwnd*
- on loss (timeout): cwnd = 2 segments

recovery (known loss)

- resend immediately on third dup ACK
- *send new data based on self-clocking idea (“fast recovery”)*

“slow start” (cwnd < ssthresh)

- (details later)

fast decrease (v2)

resetting window size to 1 is too slow

compromise:

- loss detected without timeout: divide window size by two
- loss detected with timeout: decrease to 1

exactly by two probably not important

important theoretically: *multiplicative* decrease

- will help show okay behavior with multiple flows

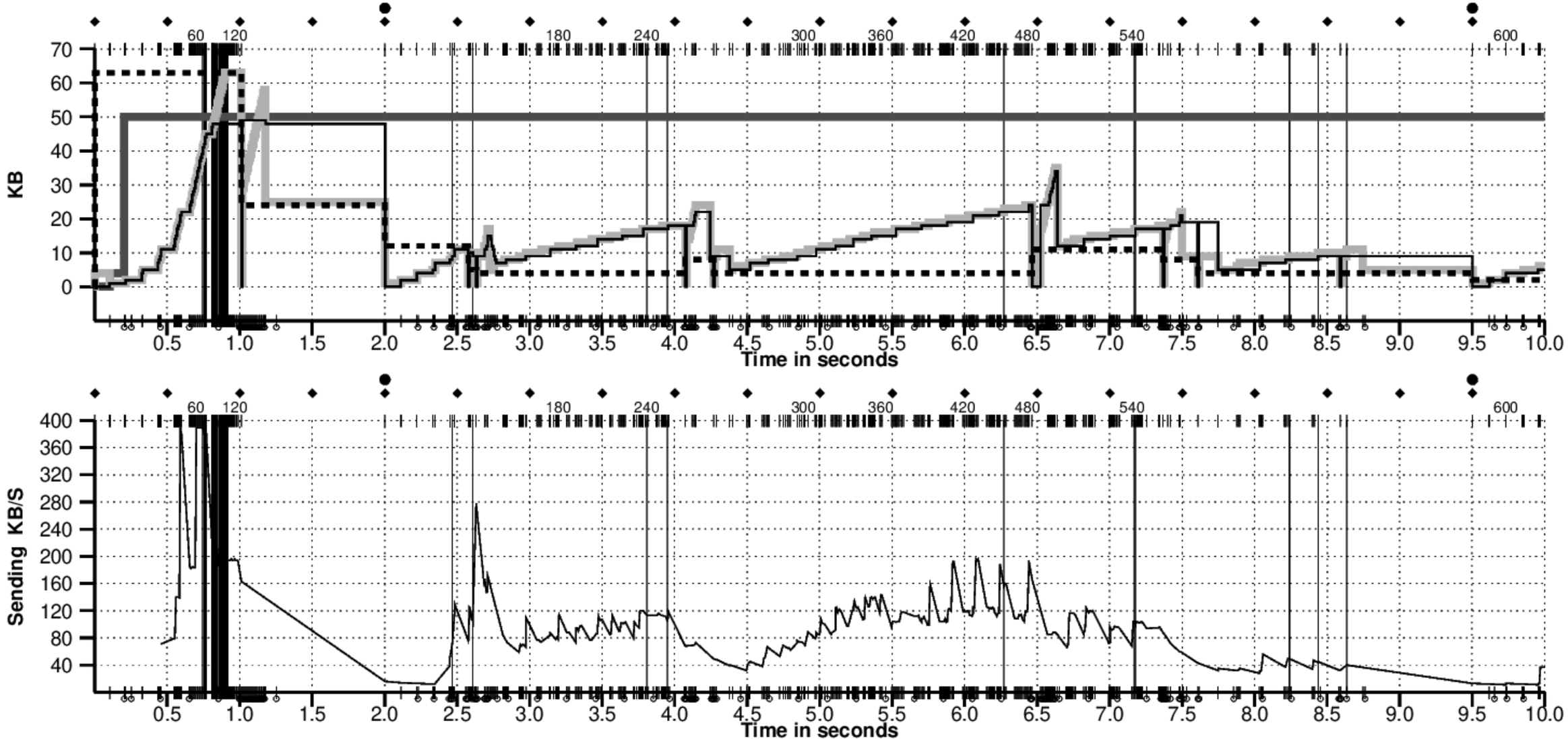
AIMD

result: additive increase + multiplicative decrease

preview: good theoretical properties for *sharing connections*

doesn’t matter much what additive/multiplicative factor is!

but: not quite what current Internet does



top thick, light-grey line = congestion window; dotted = slow start threshold

from Brakmo, O'Malley, and Peterson, "TCP Vegas: New techniques for congestion detection and avoidance"

non-congestion losses

we were ignoring non-congestion losses

suppose *1%* loss rate from transmission errors

if huge bandwidth, 50 ms RTT

with TCP heuristics (+1 packet/RTT, half on loss)…

normal window size? achieved bandwidth (pkts/sec)?

window size increases for about 100 packets, then halves

starting at window size 8:

8, 9 (17 total), 10 (27), 11 (38), 12 (50), 13 (63), 14 (77), 15 (92), 16 (>100)

$\rightarrow$ window size fluctuates from around 8 to 16

12 pkts/50 ms = 240 pkts/sec

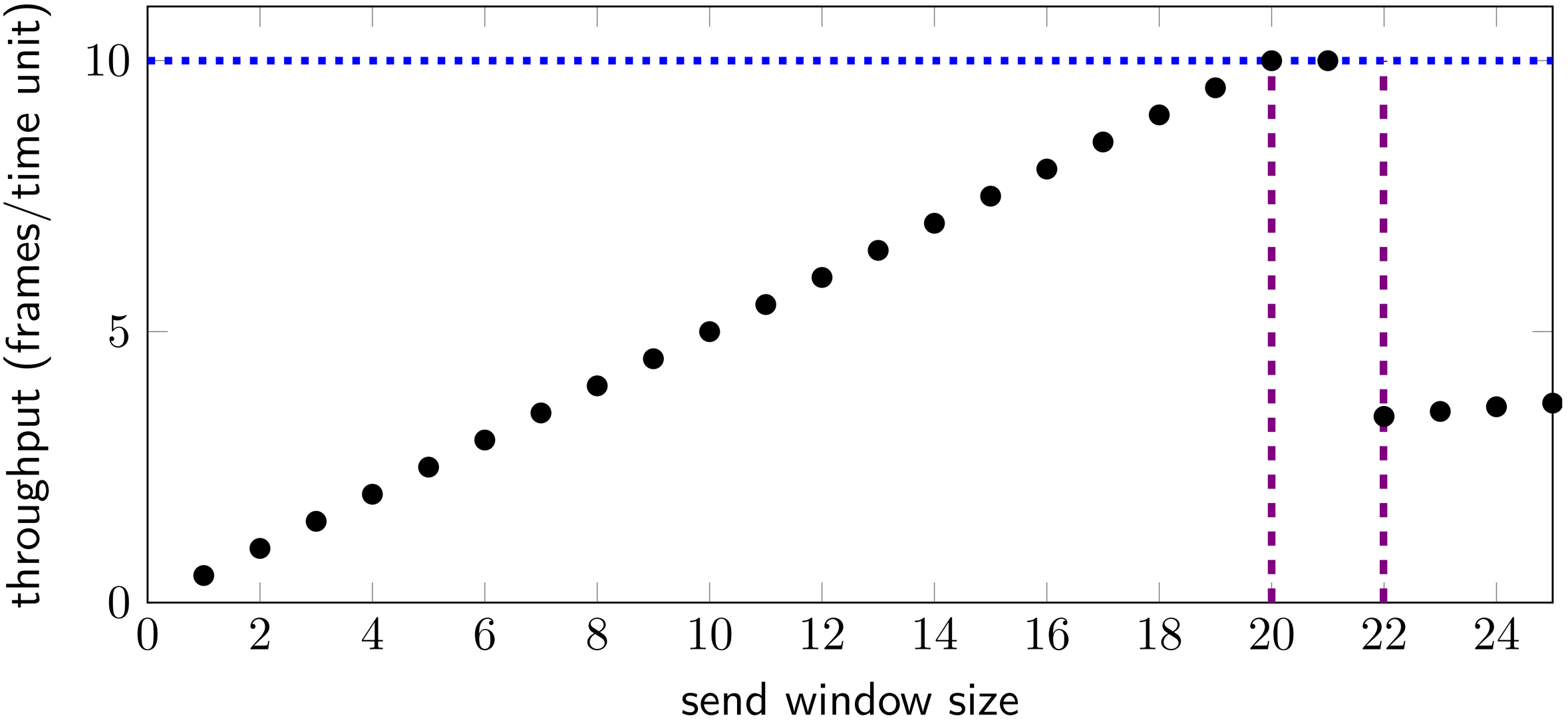
non-congestion losses and congestion control

significant non-congestion losses $\rightarrow$ very bad performance with most congestion control

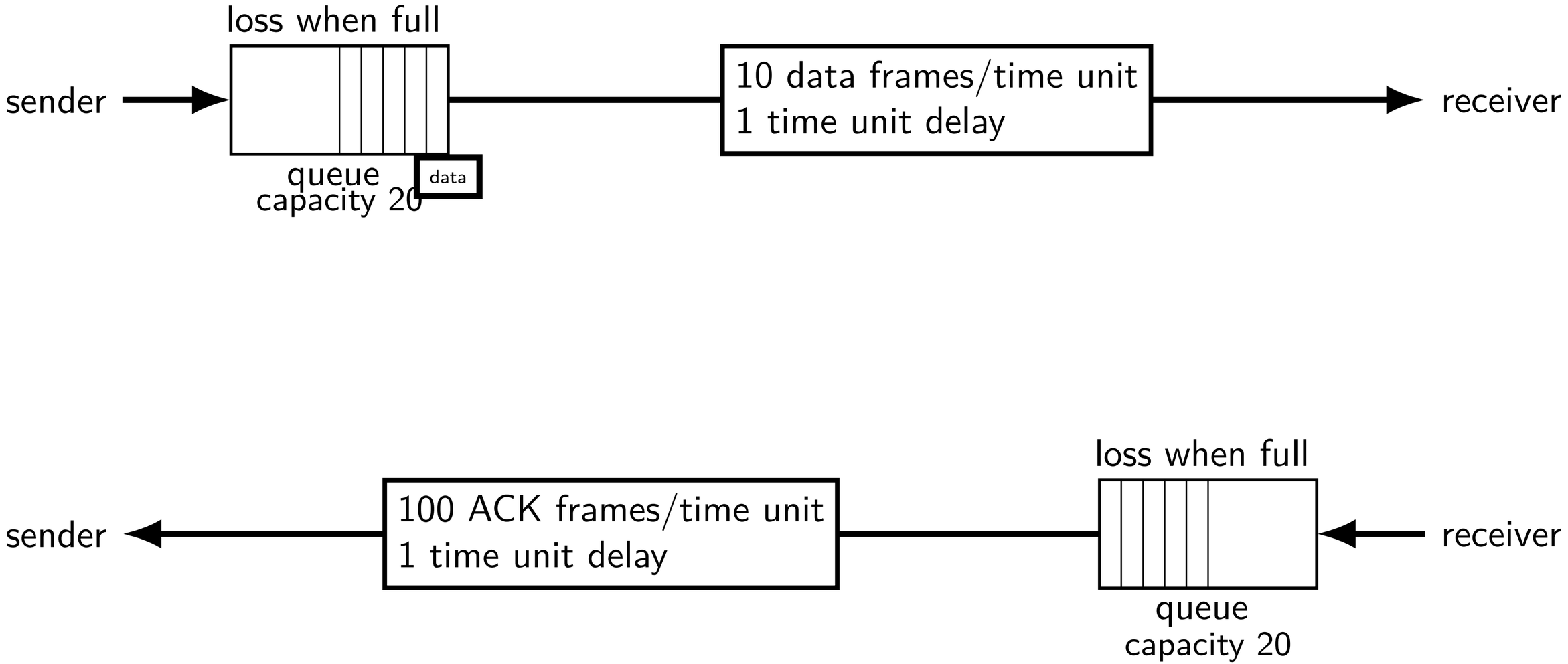
reason why wireless, etc. often does its own acknowledgements and resending

backup slides

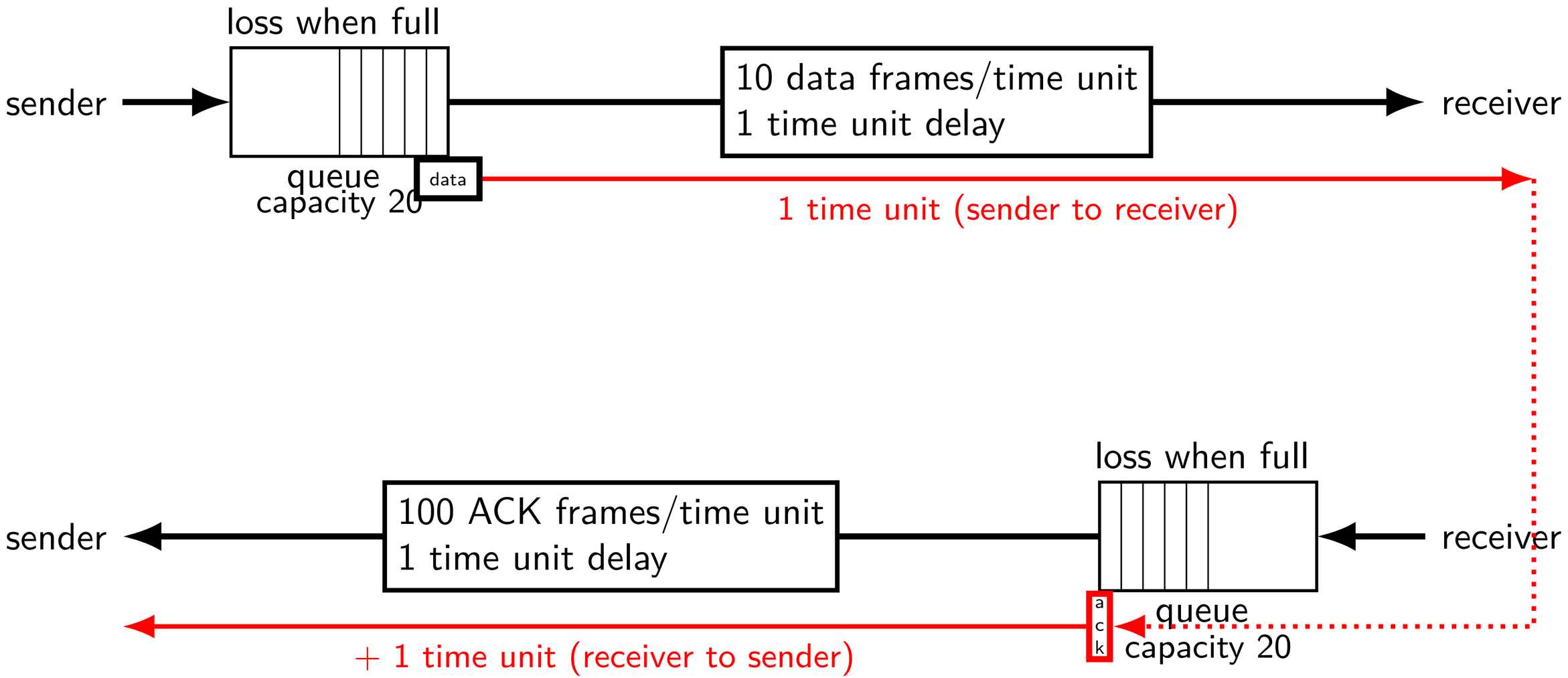
throughput and window size



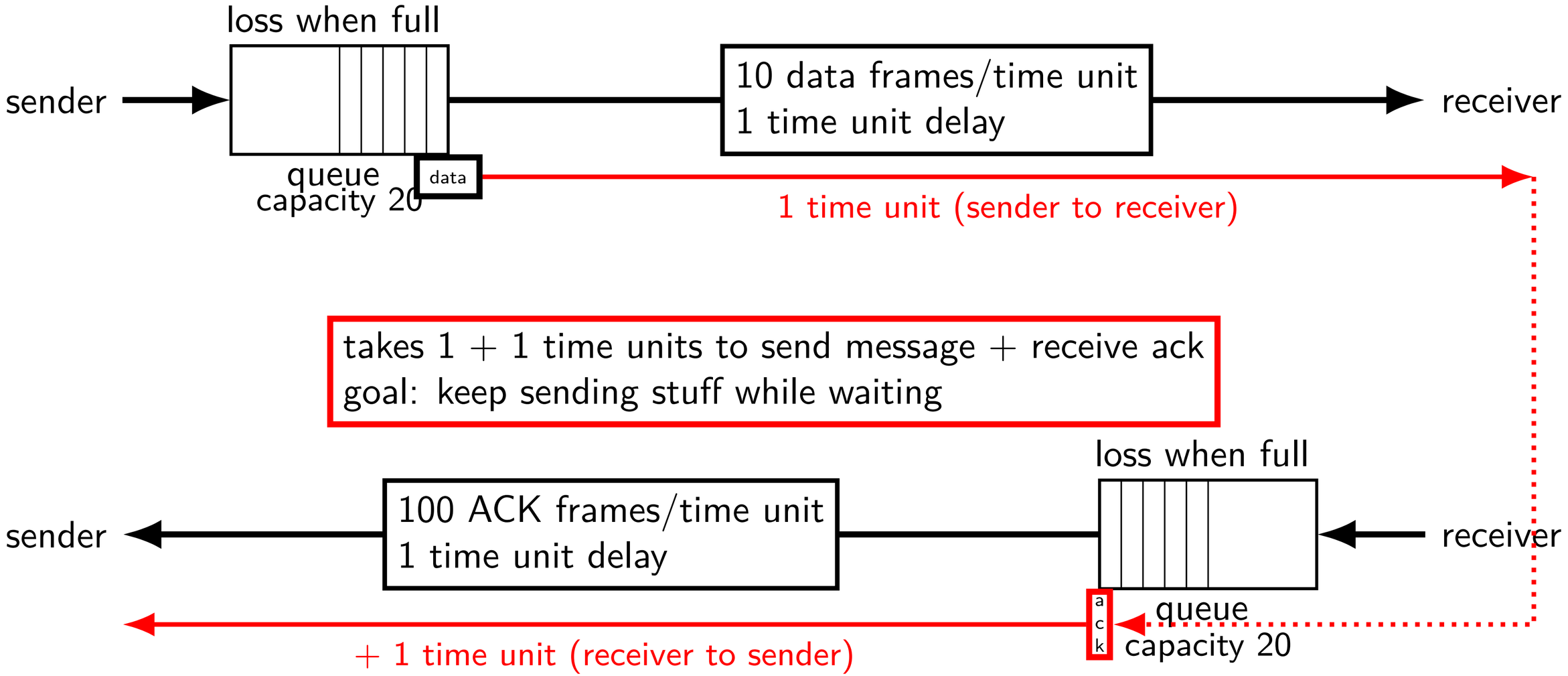
packet transit time



packet transit time



packet transit time



filling the pipe

round-trip time of 2 time units

from send data to receive ACK (assuming no queuing delay)

can send 10 data frames per time unit

= can send 20 data frames while waiting for ACK

"bandwidth-delay product"

10/time unit (banwidth) times 2 time unit (RTT = delay)

why optimal

window size $\sim$ number of packets in flight

in normal operation with window size W

- receive ACK for x (now $W - 1$ in flight)
- send packet $x + W$
- receive ACK for $x + 1$
- send packet $x + W + 1$
- ...

window size keeps W packets in flight

- increase window = more packets in flight
- decrease window = reduce packets in flight

ideal choice $\sim$ amount network can handle

number in flight on losses

window size W

let's say we lose packet x [only], sender might

- receive ACK for $x - 1$
- send packet $x + W - 1$
- receive ACK for $x, x, x, ...$
- resend packet x (guess it is lost)
- receive ACK for $x, x, x, ...$
- receive ACK for packet $x + W - 1$
- send packets $x + W$ through $x + W + W - 1$

number in flight on losses

window size W

let's say we lose packet x [only], sender might

- receive ACK for $x - 1$
- send packet $x + W - 1$
- receive ACK for $x, x, x, \ldots$
- resend packet x (guess it is lost)
- *receive ACK for $x, x, x, \ldots$*
- receive ACK for packet $x + W - 1$
- send packets $x + W$ through $x + W + W - 1$

number in flight on losses

window size W

let's say we lose packet x [only], sender might

receive ACK for $x - 1$

send packet $x + W - 1$

receive ACK for $x, x, x, ...$

resend packet x (guess it is lost)

receive ACK for $x, x, x, ...$

receive ACK for packet $x + W - 1$

send packets $x +$

lots of time when we are not sending packets

means links are underutilized

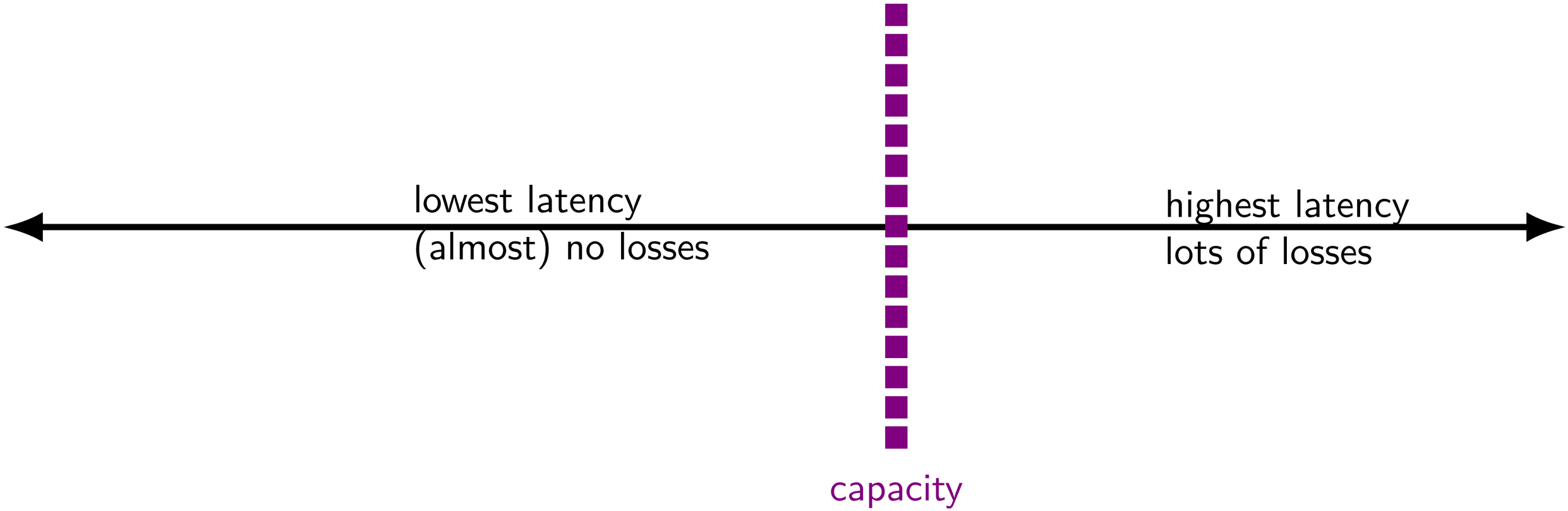
window size tweaking

window size imperfect proxy for # packets in flight

we'll ignore the difference for now

our goal for now: window size = number of packets to have in flight

finding window size empirically (1)



key insight

latency/loss rate increases when window size too big

latency/loss rate stable when window size not too big

for now, we'll focus on loss rate

- but you can do something similar with latency

try a bunch of things

try a bunch of things

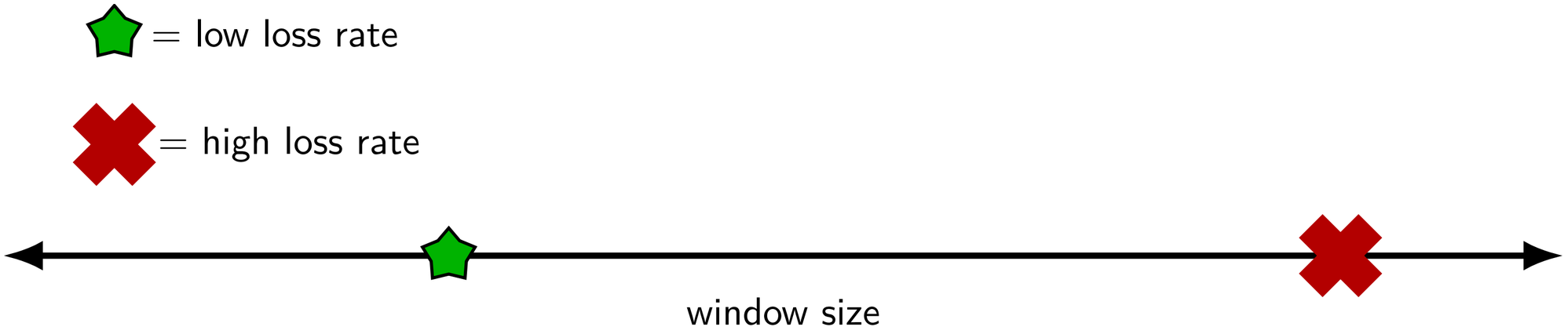

= low loss rate


= high loss rate

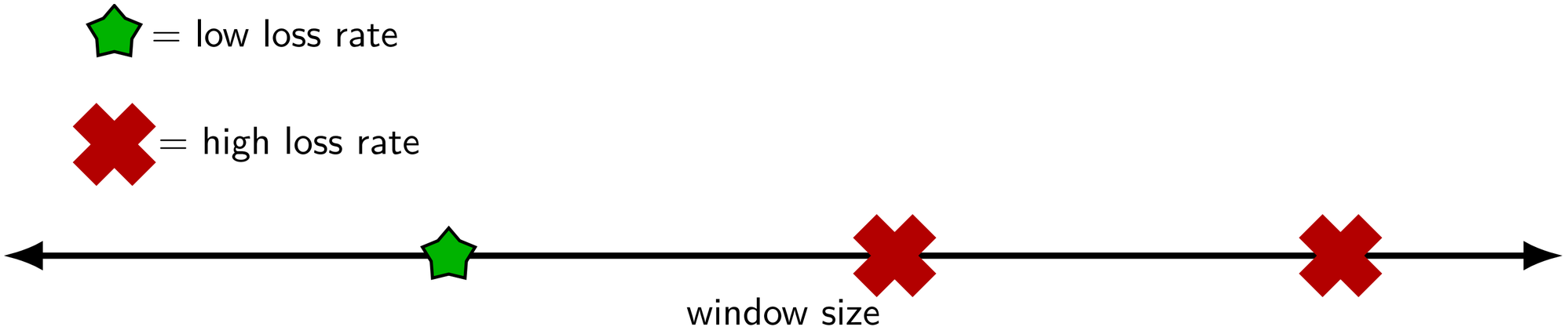


window size

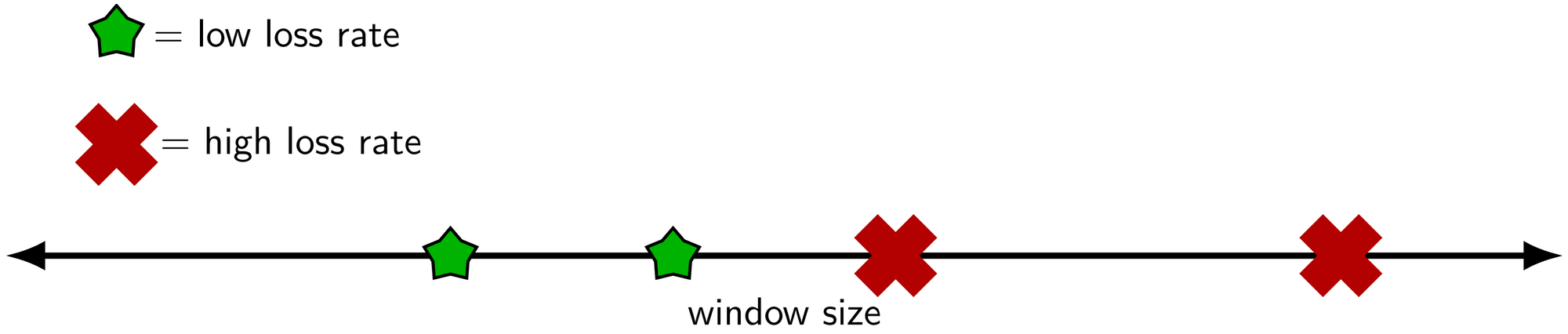
try a bunch of things



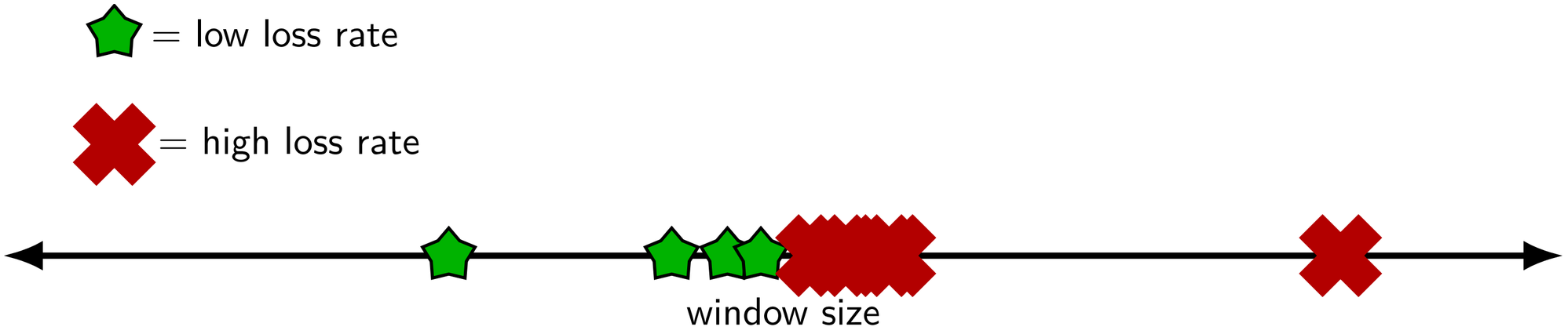
try a bunch of things



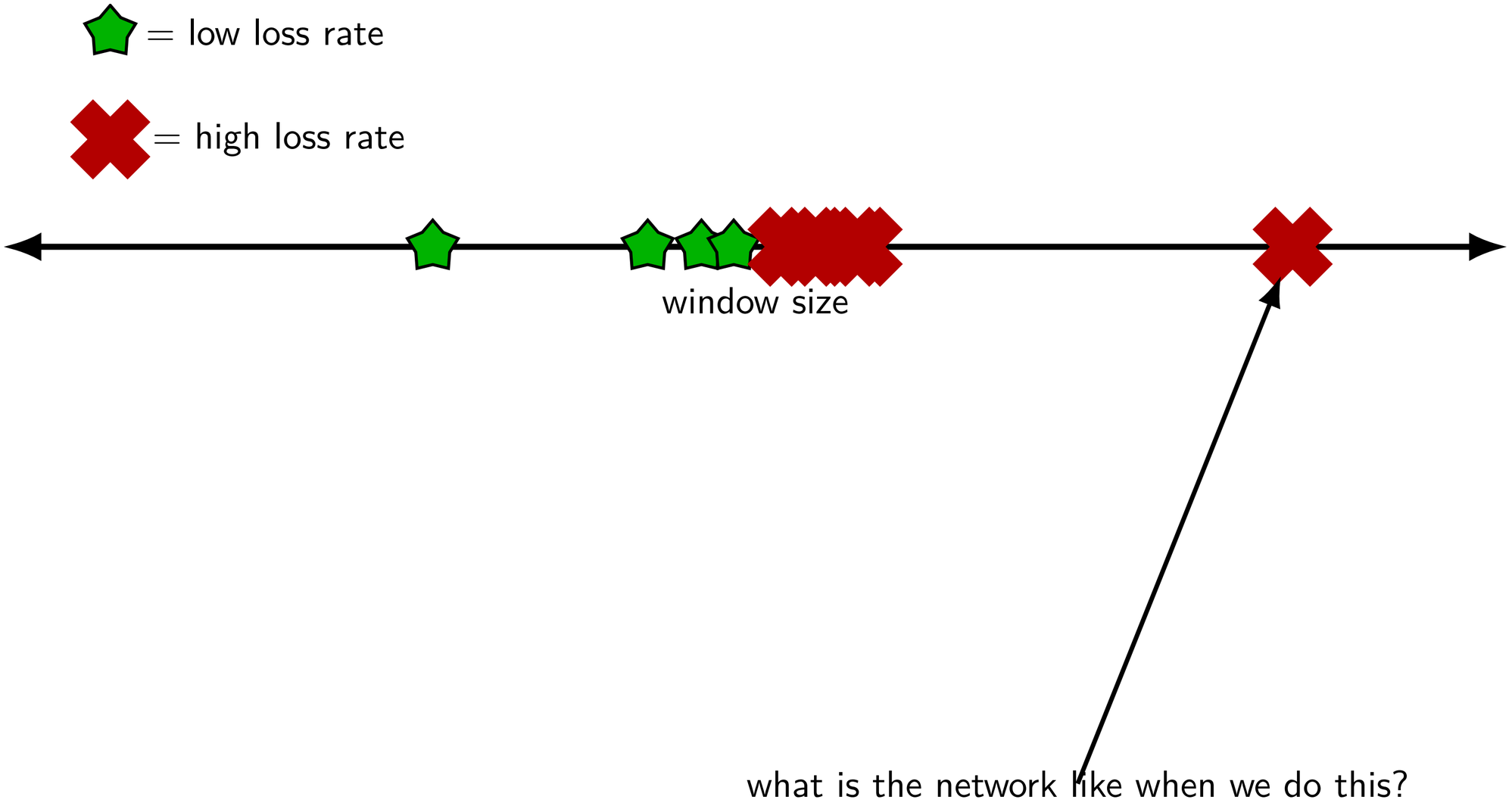
try a bunch of things



try a bunch of things



try a bunch of things



revisiting congestion collapse

Congestion Avoidance and Control

Van Jacobson*

University of California
Lawrence Berkeley Laboratory
Berkeley, CA 94720
van@helios.ee.lbl.gov

In October of '86, the Internet had the first of what became a series of 'congestion collapses'. During this period, the data throughput from LBL to UC Berkeley (sites separated by 400 yards and three IMP hops) dropped from 32 Kbps to 40 bps. Mike Karels[1] and I were fascinated by this sudden factor-of-thousand drop in bandwidth and embarked on an investigation of why things had gotten so bad. We wondered, in particular,

fixes from Jacobson's 1987 paper

1. round-trip-time variance estimation
2. exponential retransmit timer backoff
3. slow-start
4. more aggressive receiver ack policy
5. dynamic window sizing on congestion
6. Karn's clamped retransmit backoff
7. fast retransmit

the overloaded switch

let's say switch can handle 50 packets/second

but has:

- 100 packets/second from test flow sending as fast as it can
- 10 packets/second from other session

expected *loss rate* (% packets lost)?

expected % test flow packets lost?

expected other session packets lost?

modeling who gets dropped

it kinda does matter…

sending in big bursts or spread out (“pacing”)?

- bursts can overload queues even though average rate is low

how switch’s queue works?

- queue size (handling bursts), way to choose what to drop

random or fixed intervals between sending?

but we’ll simplify, assuming—

- a flow’s arrivals are randomly spaced
- drops hit packets at random
- queue is “pretty big”

modeling who gets dropped

it kinda does matter…

sending in big bursts or spread out (“pacing”)?

- bursts can overload queues even though average rate is low

how switch’s queue works?

- queue size (handling bursts), way to choose what to drop

random or fixed intervals between sending?

but we’ll *simplify*, assuming—

- a flow’s arrivals are randomly spaced
- drops hit packets at random
- queue is “pretty big”

the overloaded switch

let's say switch can handle 50 packets/second

but has:

- 100 packets/second from test flow (checking window size)
- 10 packets/second from other session

expected *loss rate* (% packets lost)? $\frac{100+10-50}{100+10} = 54\%$

expected % test flow packets lost? 54%

expected % other session packets lost? 54%

… but I missed something

a virtuous cycle

what is other session going to when 54% of its packets are lost?

probably resend them

what about when resent packets are lost?

probably resent again

if other session doesn't slow down, then...

10 pkt/s $\rightarrow 10 + 54\% \cdot 10 + 54\%^2 \cdot 10 \ldots \approx 22$ pkt/s

the overloaded switch (revised)

let's say switch can handle 50 packets/second

but has:

- 100 packets/second from test flow (checking window size)
- 10 packets/second from other session $\rightarrow$ 22 with resends

expected *loss rate* (% packets lost)? $\frac{100+22-50}{100+22} = 59\%$

expected % test flow packets lost? 59%

expected % other session packets lost? 59%

- means that 22 pkt/sec is slight underestimate
- though realistically other session should slow down

aside: latency (1)

need one round-trip time (RTT) to detect loss

- probably from duplicate ACK
- if detecting via timeout, probably longer

so need 1.4 RTTs (detecting loss 1.4 times) extra time

mean latency $= \frac{1.4\text{RTTs}}{0.5\text{RTTs}}$ times normal $= 2.8$ times normal

aside: high-percentile latency

59% packet loss

about 10% of time need more than 4 retransmissions

about 5% of the time need more than 5 retransmissions

about 1% of the time need more than 8 retransmissions

sliding windows and retransmissions

we were assuming that other session doesn't slow down

sliding window approach slows down on losses given same window size

sliding window throughput collapse

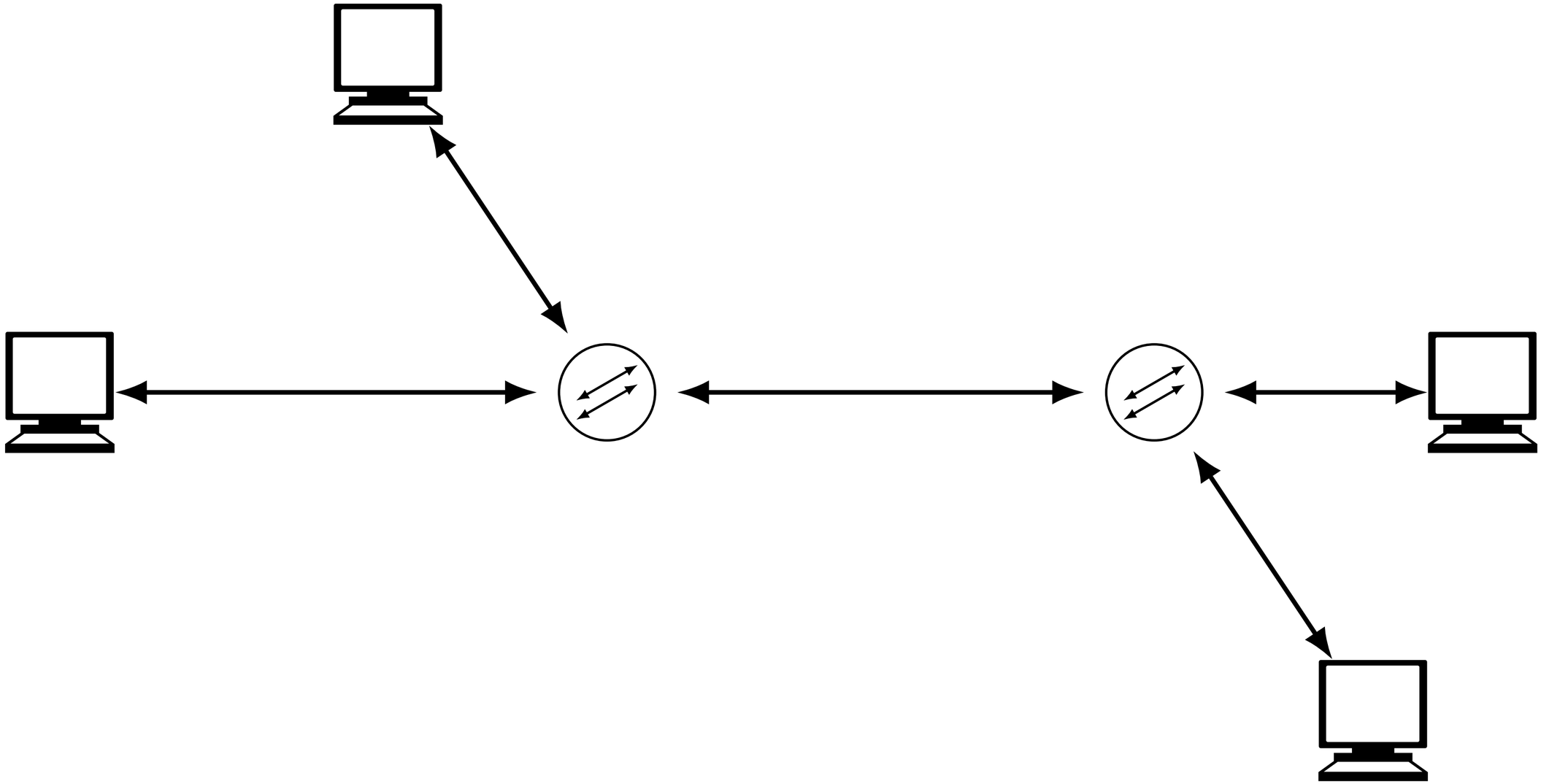
let's say doing sliding window with 100 packet window

if 1% of the time, we need to resend a packet 8 times, then

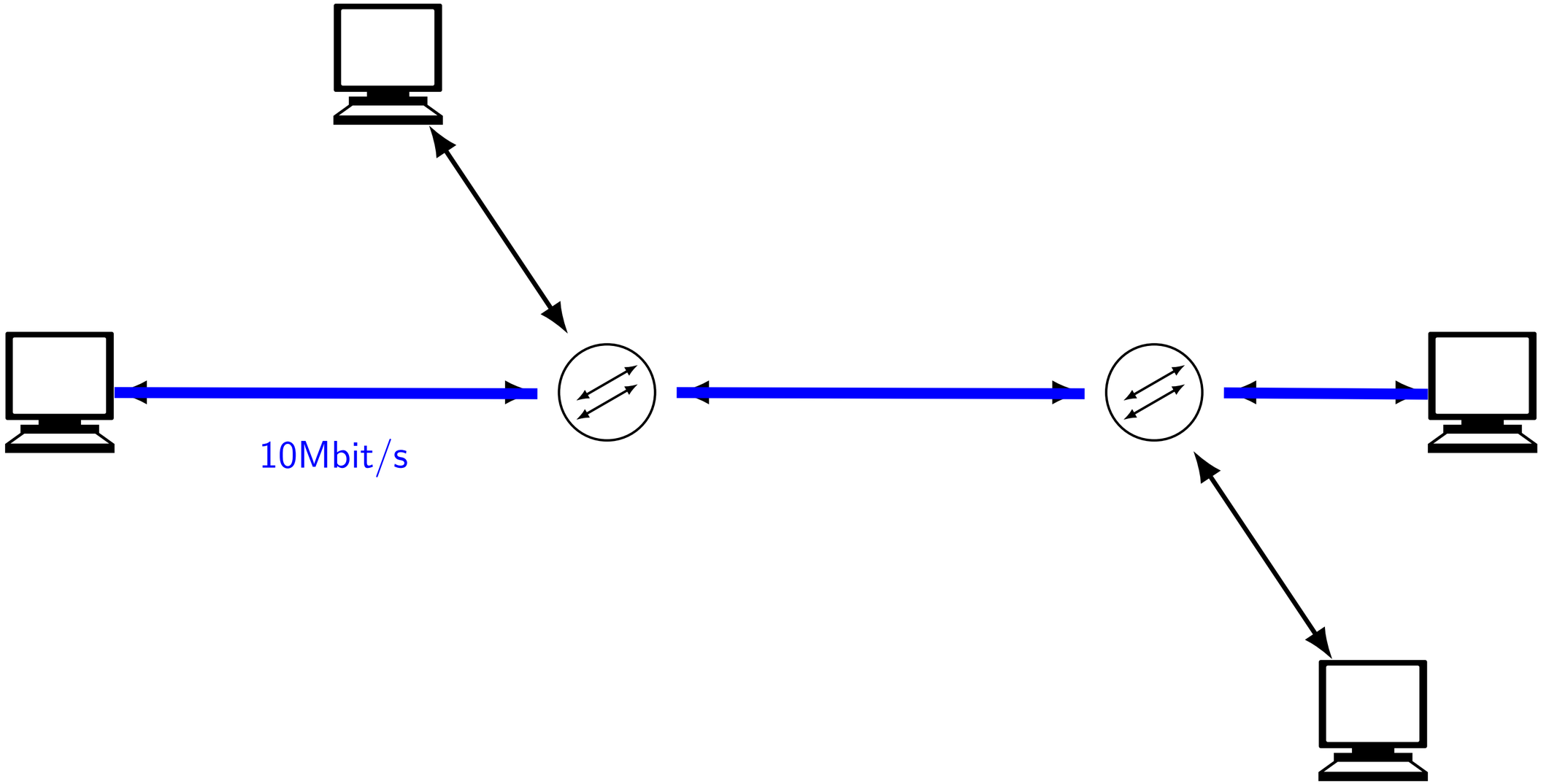
probably need around 8 RTTs to send all 100 packets in window

$\approx$ 8 times slower with same window size

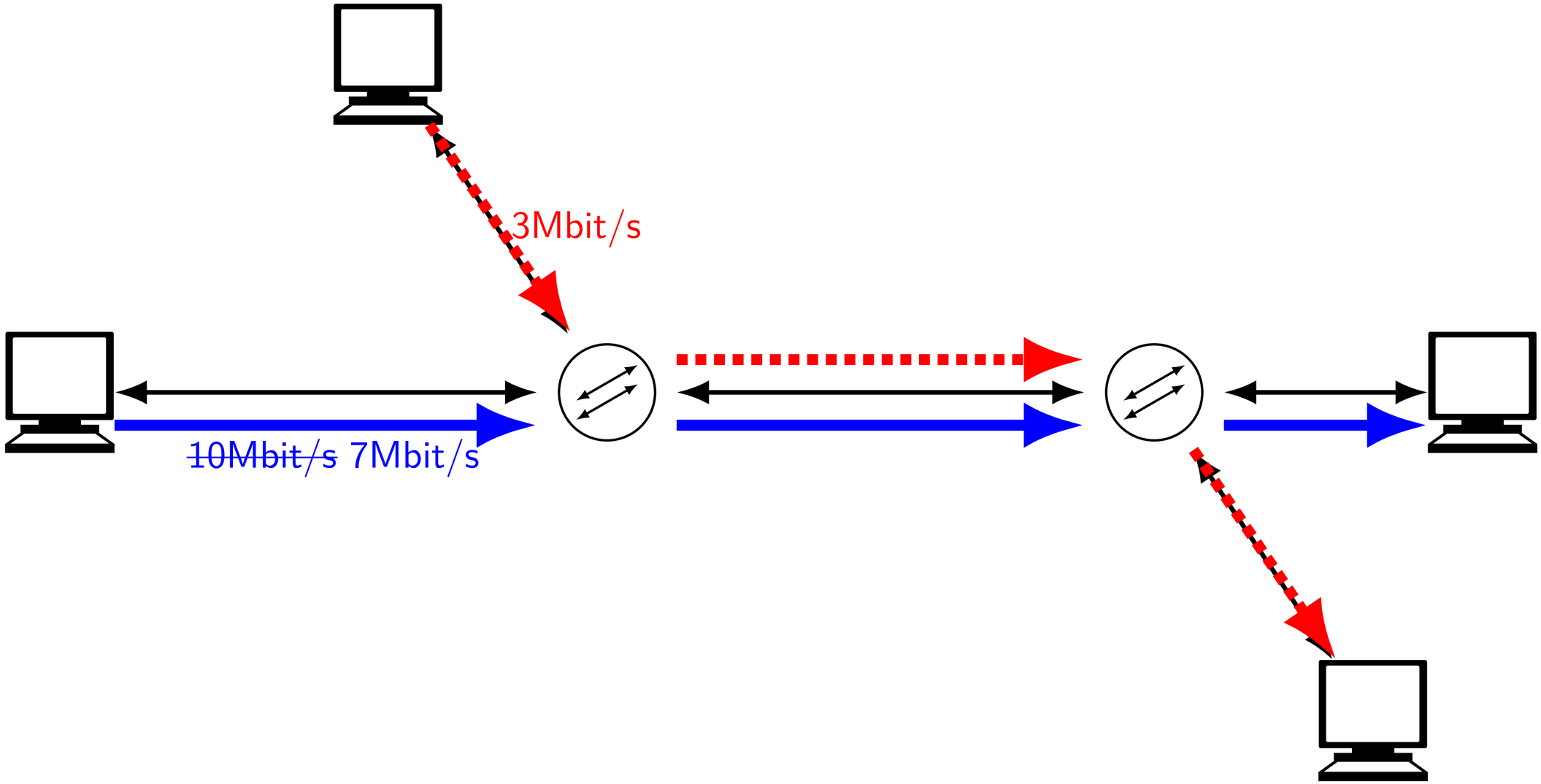
changing cross-traffic



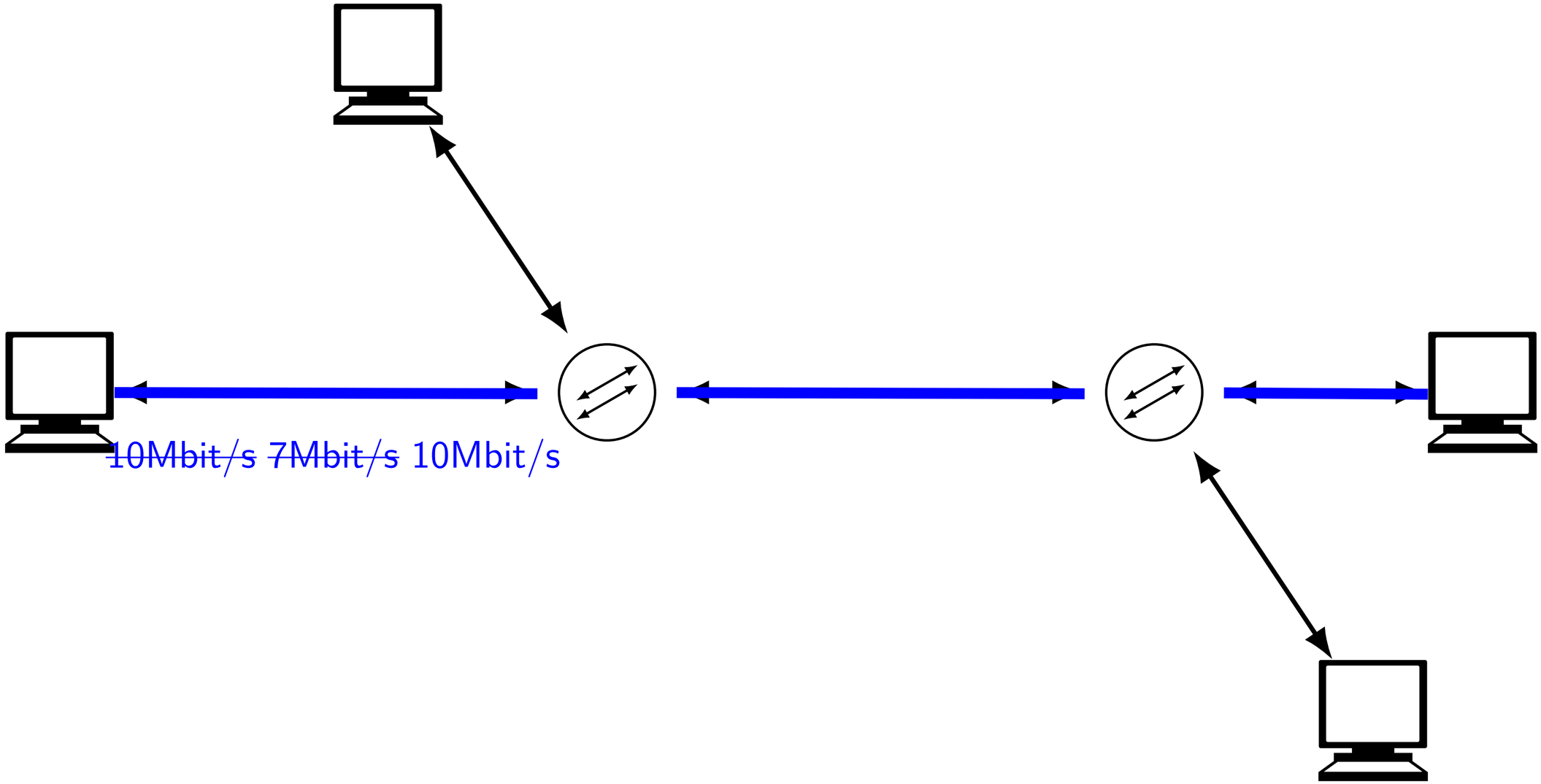
changing cross-traffic



changing cross-traffic



changing cross-traffic



adapting to cross-traffic

available bandwidth will change

previous example: 3Mbit lost/added from other flow

need to adapt to lost bandwidth

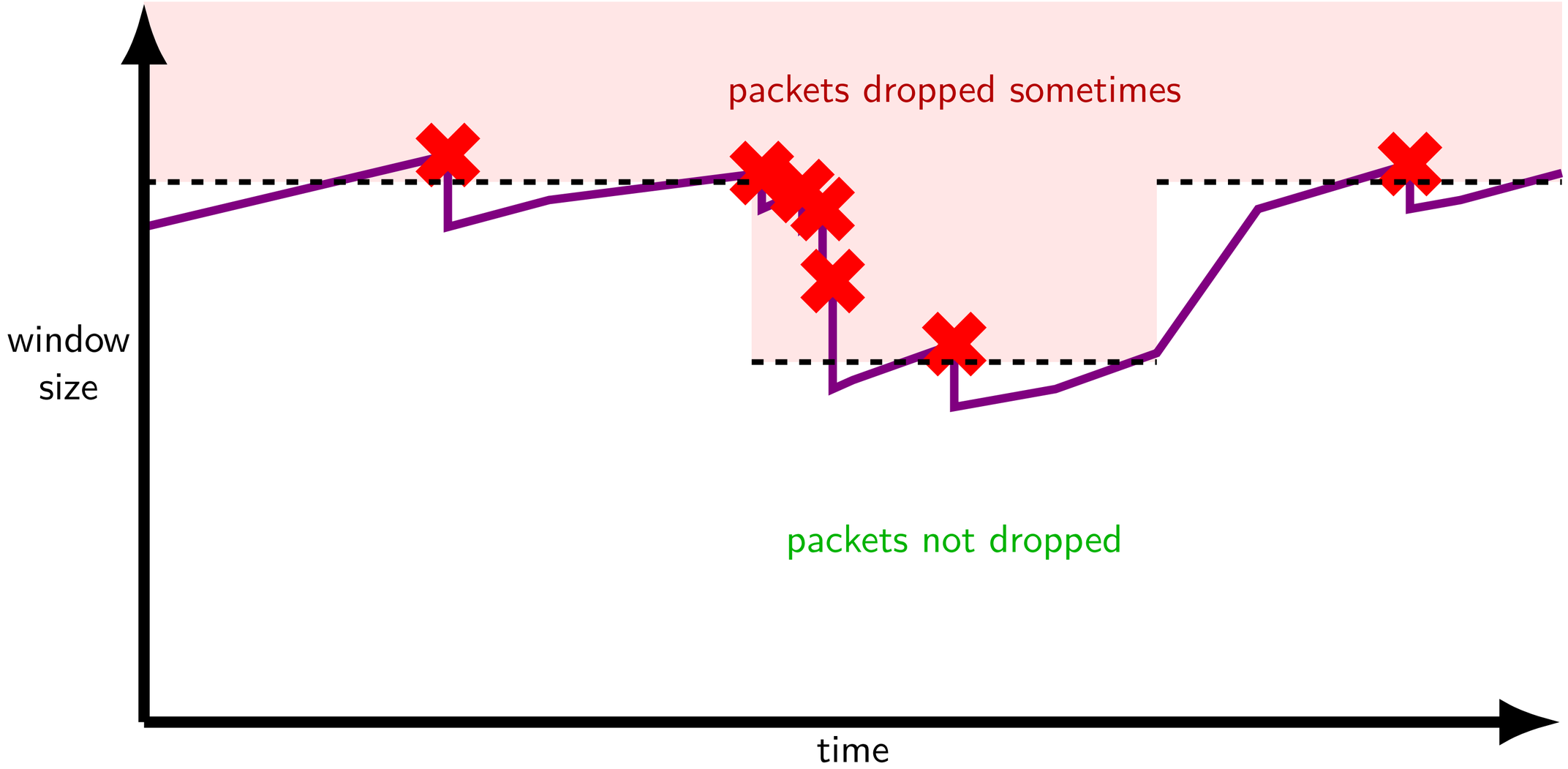
need to detect new available bandwidth

other flow's bandwidth?

for now, we'll pretend other flows don't react to us

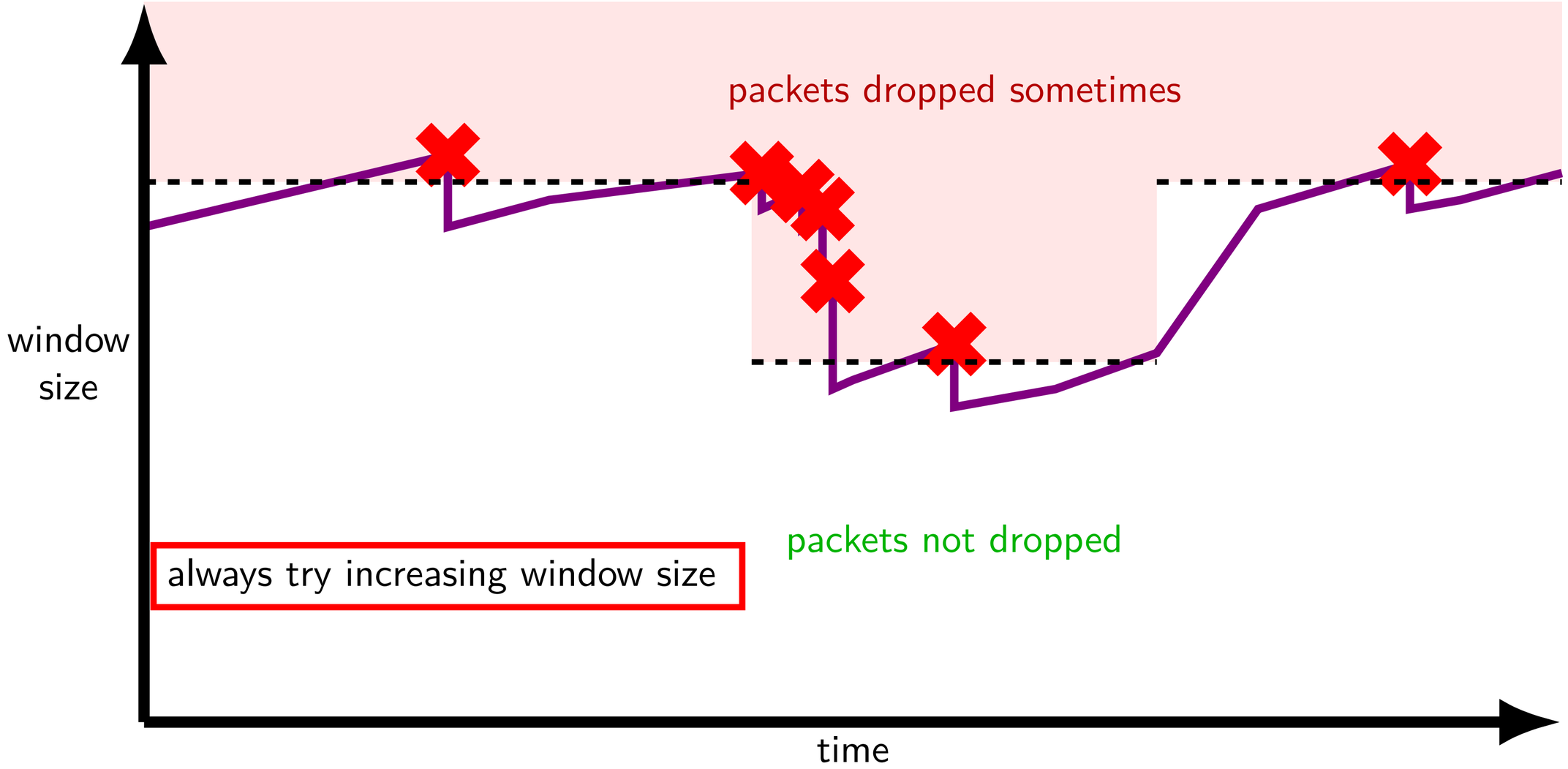
later topic: what happens when both reacting?

window size experimenting

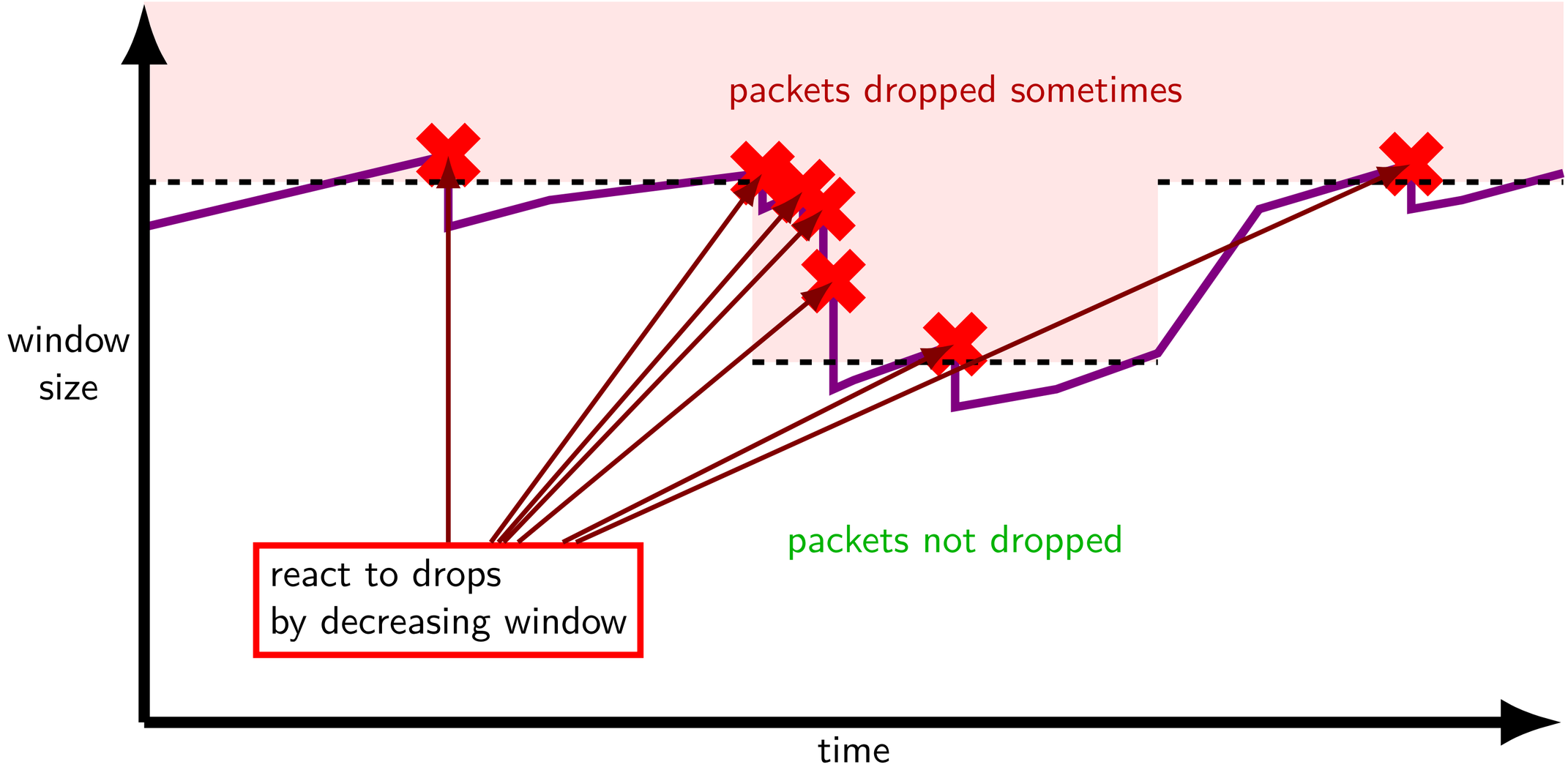


window size experimenting

window size experimenting



window size experimenting



increase/decrease strategy

default to increasing window size

react to packet drops by decreasing window size

- assumption: few “non-congestion” packet losses

big topic: how fast to do each?

questions to help decide that:

- what happens if we increase too fast? too slow?
- what happens if we decrease too fast? too slow?

increase/decrease strategy

default to increasing window size

react to packet drops by decreasing window size

assumption: few "non-congestion" packet losses

big topic: how fast to do each?

questions to help decide that:

what happens if we increase too fast? too slow?

what happens if we decrease too fast? too slow?

slow increase

want to increase *slowly* to avoid overload

original TCP: +1 packet/round trip time

+1 certainly not optimal choice, but okay heuristic

important theoretically: approx. *additive* increase

- helps ensure good behavior with multiple connections
- (we'll talk later about why)

exercise: convergence time (1)

suppose: 50 ms round trip time

initially sending at 600 packets/second

≈ 0.9Mbyte/sec with 1500 byte packets

optimal rate is 10000 packets/second

≈ 15Mbyte/sec with 1500 byte packets

‘standard’ TCP increase of 1 packet/RTT

how long to get there?

exercise: convergence time (1)

suppose: 50 ms round trip time

initially sending at 600 packets/second

≈ 0.9Mbyte/sec with 1500 byte packets

optimal rate is 10000 packets/second

≈ 15Mbyte/sec with 1500 byte packets

'standard' TCP increase of 1 packet/RTT

how long to get there?

current: 30 packets/RTT (= window size 30)

need to get to: 500 packets/RTT

will take $500 - 30 = 470$ round trips $\approx$ 23500 ms $\approx$ 24 s

fixing bad convergence time

TCP's additive increase is very slow for "high bandwidth-delay" networks

two things make this better:

not in additive increase mode at start of connection

- "slow start" we'll talk about later

more adaptive increase for modern TCP variants

- e.g. FAST TCP, CUBIC TCP, ...
- heuristics to increase faster when appropriate

heurstic: fast decrease (v1)

want to decrease quickly to get out of overload

original TCP heuristic: reset window to (minimum) 2 segments

TCP Tahoe (1)

first version of TCP congestion control

track two variables

- `cwnd` = congestion window
- `ssthresh` = 'slow start threshold'

three modes of operation:

congestion avoidance (cwnd $\geq$ ssthresh; no known loss)

recovery (known loss)

"slow start" (cwnd < ssthresh)

TCP Tahoe (2)

congestion avoidance (cwnd $\geq$ ssthresh; no known loss)

- represents "steady state" — our focus for now
- no loss: cwnd += 1 segment per round-trip-time
- on loss: cwnd = ~ 2 segments; ssthresh = cwnd/2

recovery (known loss)

- retransmit packet after third duplicate ACK of previous packet
- (or timeout)

"slow start" (cwnd < ssthresh)

- intuition: not in "steady state" yet
- want to get there quickly, but with overloading network
- we'll give details later

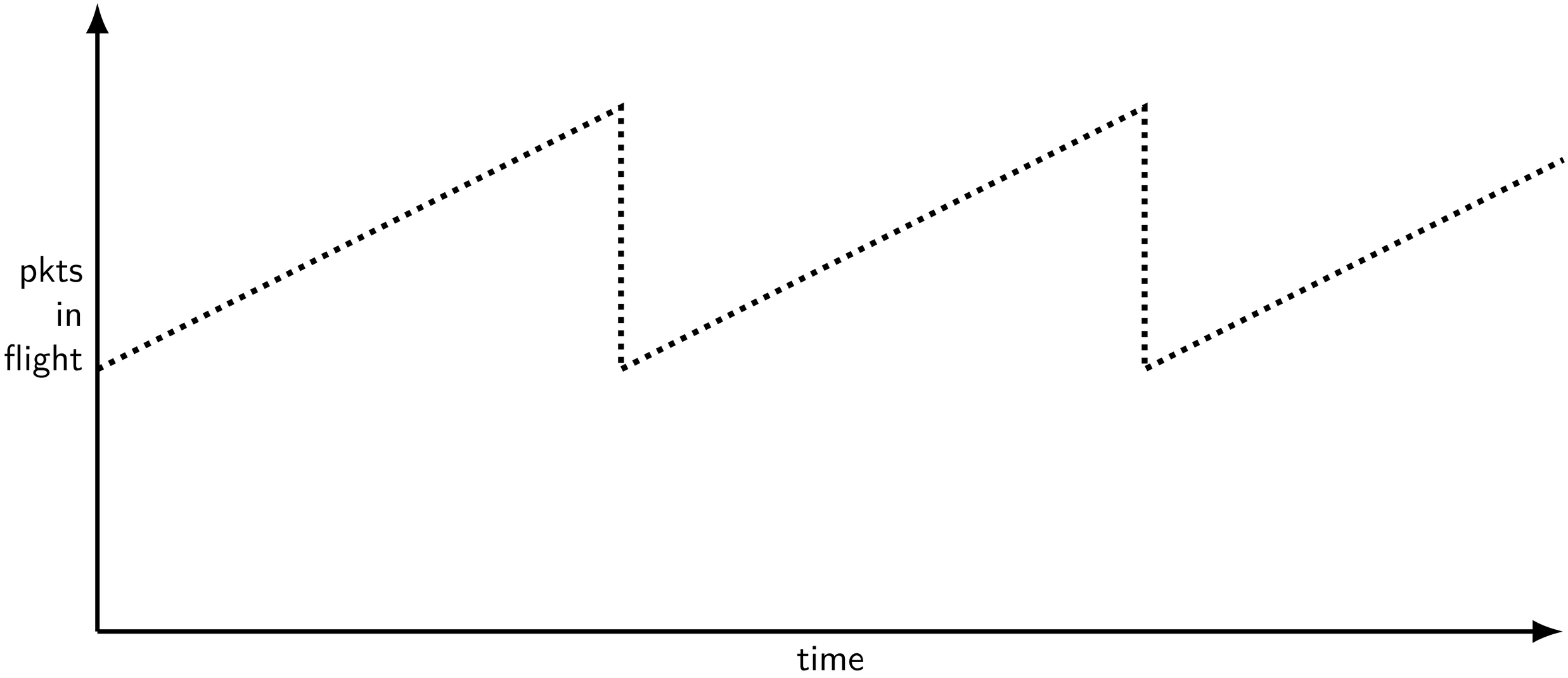
plan

really don’t want to reset to beginning on loss

- even if we have a way to get to good size quickly

would like to do less extreme decrease: cwnd = cwnd/2

sawtooth pattern



Linux's ss command

```
$ ss --info --numeric --tcp state established
...
Recv-Q  Send-Q   Local Address:Port          Peer Address:Port
0       2583232  128.143.71.27:5201          128.143.63.240:55926
cubic wscale:7,7 rto:204 rtt:1.891/0.125 ato:40 mss:1448 pmtu:1500
rcvmss:536 advmss:1448 cwnd:309 ssthresh:64 bytes_sent:152936312
bytes_acked:152698840 bytes_received:37 segs_out:105620 segs_in:7024
data_segs_out:105619 data_segs_in:1 send 1.89Gbps lastrcv:1312
pacing_rate 2.27Gbps delivery_rate 996Mbps delivered:105456
busy:1308ms unacked:164 rcv_space:14600 rcv_ssthresh:64076
notsent:2345760 minrtt:0.219
```

`cubic` = which congestion control algorithm

`cwnd` = 309 segments; `ssthresh` = 64 segments

cwnd = 309 segments $\times$ 1488B max segment size = 437 KiB

packet rate $\approx$ cwnd / rtt = 224 MiB/sec = 1879 Mbit/s

handling steady state

most of the time we should be at approx. correct window size

want to focus on how we react to changes

still going to use “experimentation” idea

fixes from Jacobson's 1987 paper

1. round-trip-time variance estimation
2. exponential retransmit timer backoff
3. slow-start
4. more aggressive receiver ack policy
5. *dynamic window sizing on congestion*
6. Karn's clamped retransmit backoff
7. *fast retransmit*

TCP (New)Reno

base for ‘modern’ TCP (changes from Tahoe highlighted)

congestion avoidance (cwnd $\geq$ ssthresh; no known loss)

- no loss: cwnd += 1 segment per round-trip-time
- *on loss (dup ACK): cwnd = cwnd/2; ssthresh = new cwnd*
- on loss (timeout): cwnd = 2 segments

recovery (known loss)

- resend immediately on third dup ACK
- *send new data based on self-clocking idea (“fast recovery”)*

“slow start” (cwnd < ssthresh)

- (details later)

fast decrease (v2)

resetting window size to 1 is too slow

compromise:

- loss detected without timeout: divide window size by two
- loss detected with timeout: decrease to 1

exactly by two probably not important

important theoretically: *multiplicative* decrease

- will help show okay behavior with multiple flows

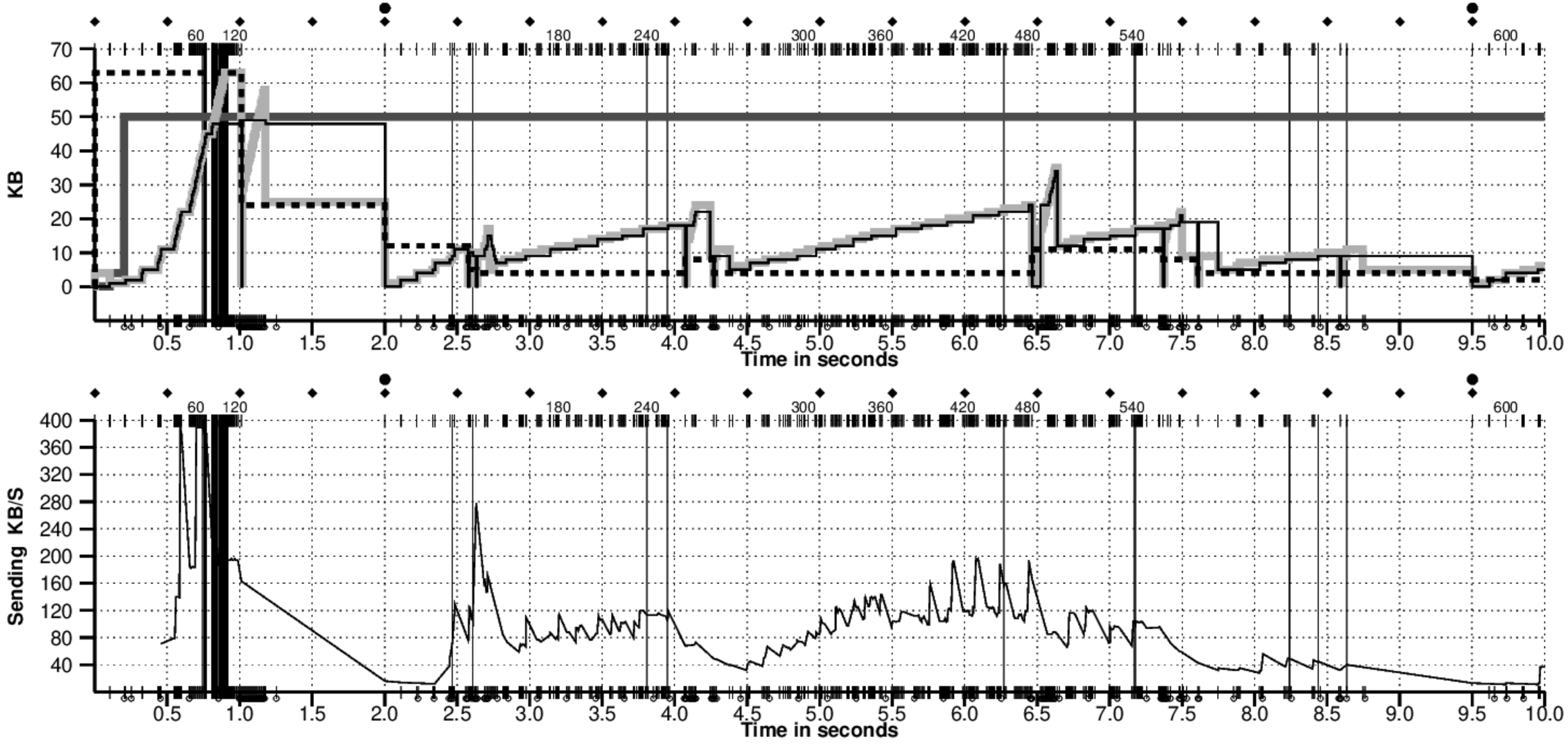
AIMD

result: additive increase + multiplicative decrease

preview: good theoretical properties for *sharing connections*

doesn’t matter much what additive/multiplicative factor is!

but: not quite what current Internet does



top thick, light-grey line = congestion window; dotted = slow start threshold

from Brakmo, O'Malley, and Peterson, "TCP Vegas: New techniques for congestion detection and avoidance"

non-congestion losses

we were ignoring non-congestion losses

suppose *1%* loss rate from transmission errors

if huge bandwidth, 50 ms RTT

with TCP heuristics (+1 packet/RTT, half on loss)…

normal window size? achieved bandwidth (pkts/sec)?

window size increases for about 100 packets, then halves

starting at window size 8:

8, 9 (17 total), 10 (27), 11 (38), 12 (50), 13 (63), 14 (77), 15 (92), 16 (>100)

$\rightarrow$ window size fluctuates from around 8 to 16

12 pkts/50 ms = 240 pkts/sec

non-congestion losses and congestion control

significant non-congestion losses $\rightarrow$ very bad performance with most congestion control

reason why wireless, etc. often does its own acknowledgements and resending